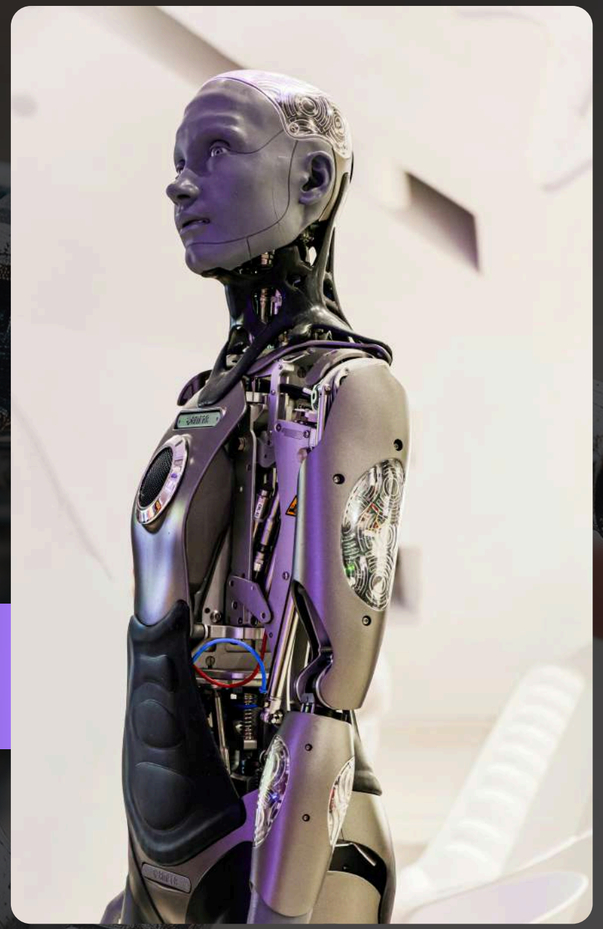


UNIT 2

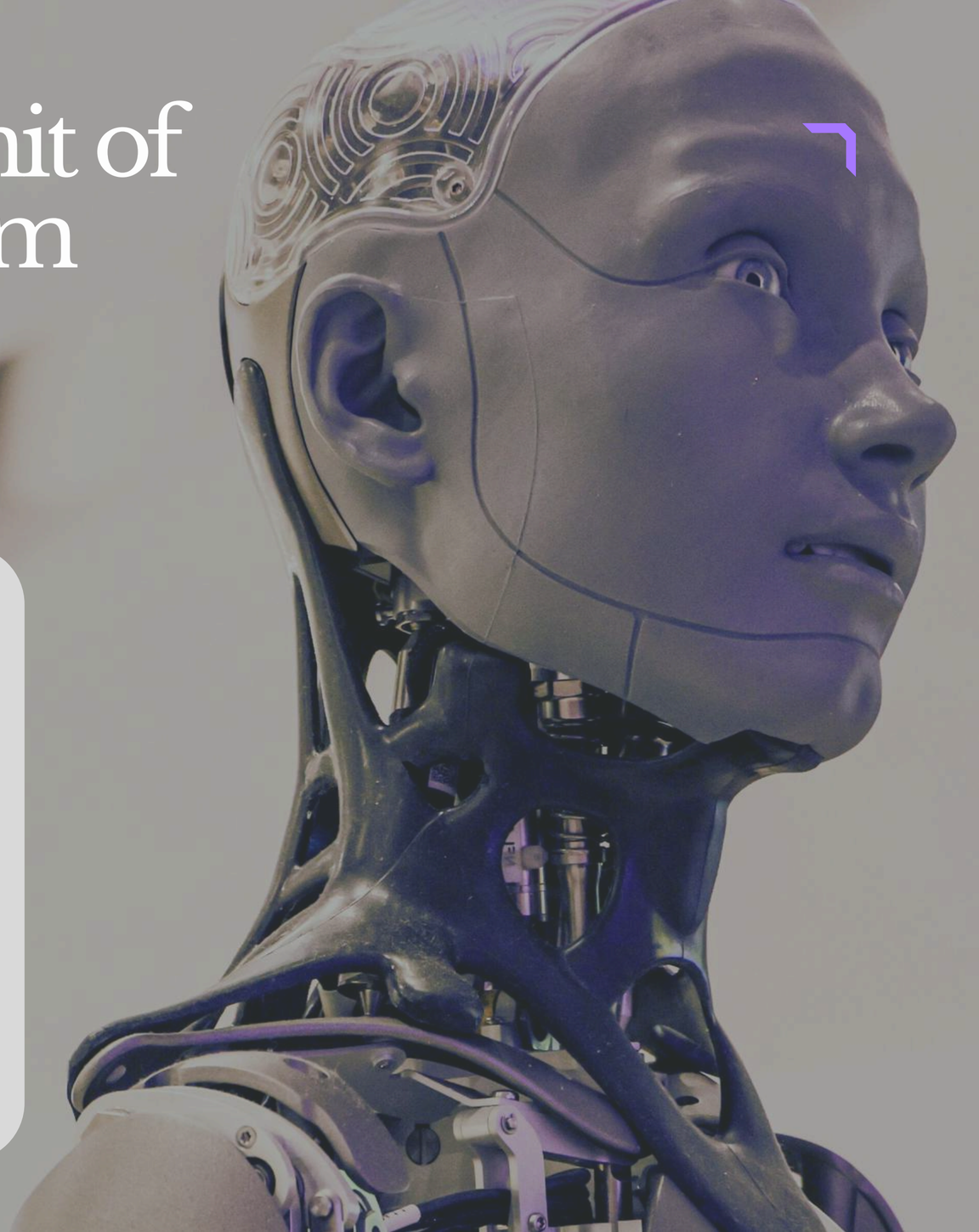
Deep Learning

Deep Neural Network

Neuron: Fundamental Unit of Brain & Nervous System

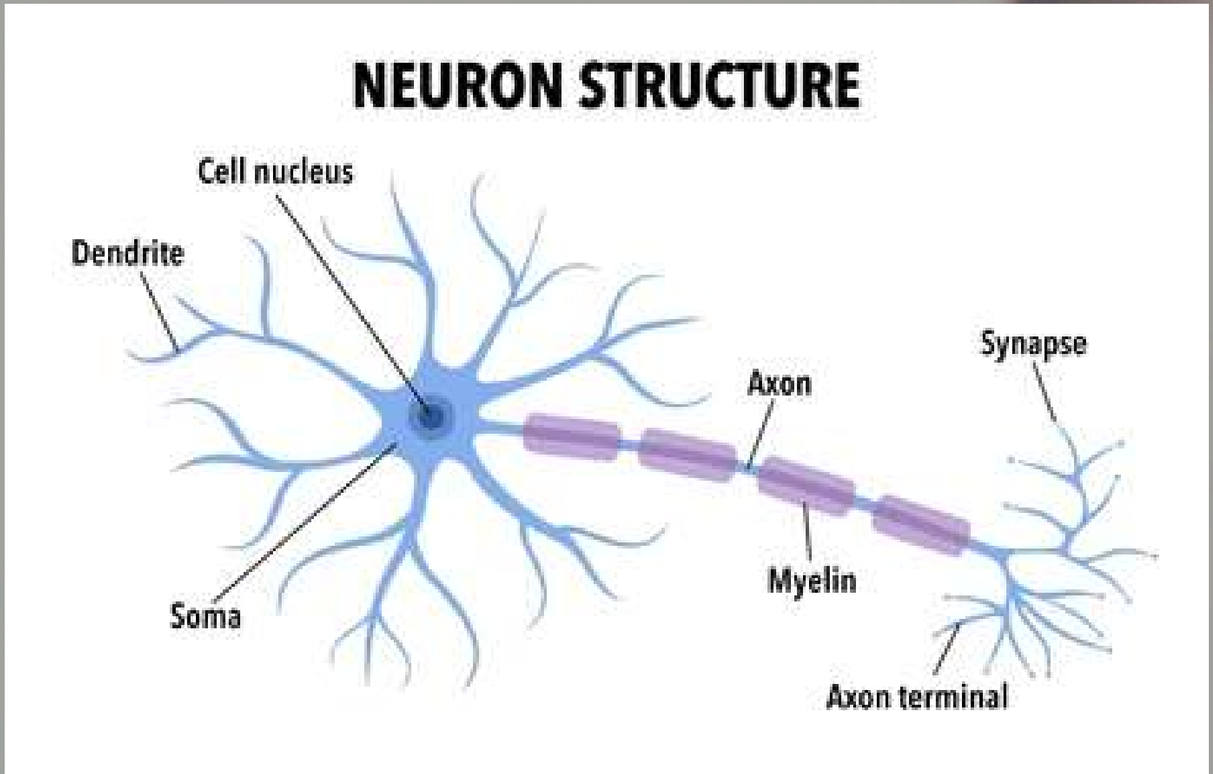
Definition

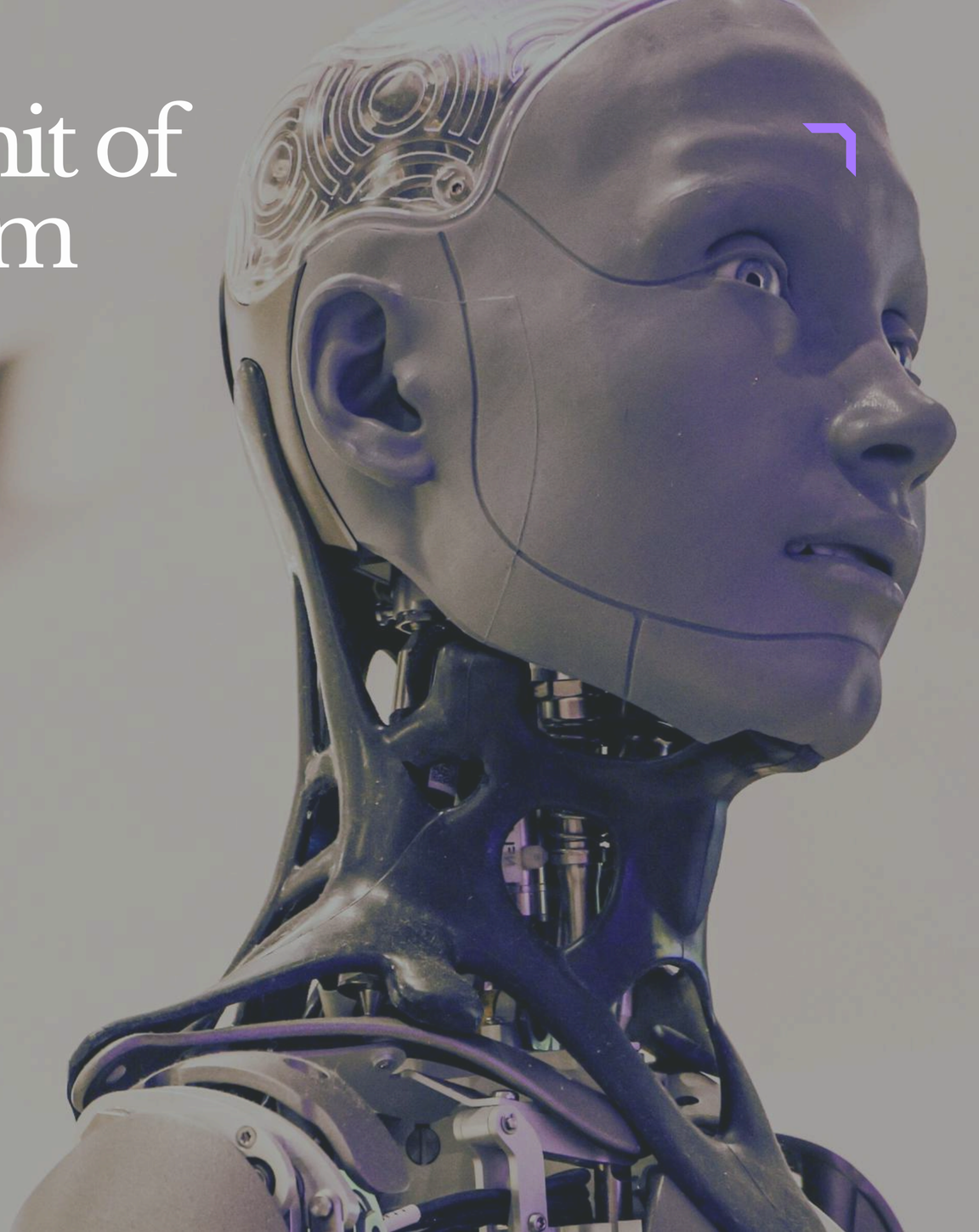
- Neuron is the basic structural and functional unit of the human brain and nervous system.
- Responsible for processing and transmitting information.
- Signals travel through electrical and chemical signals.



Neuron: Fundamental Unit of Brain & Nervous System

Diagram



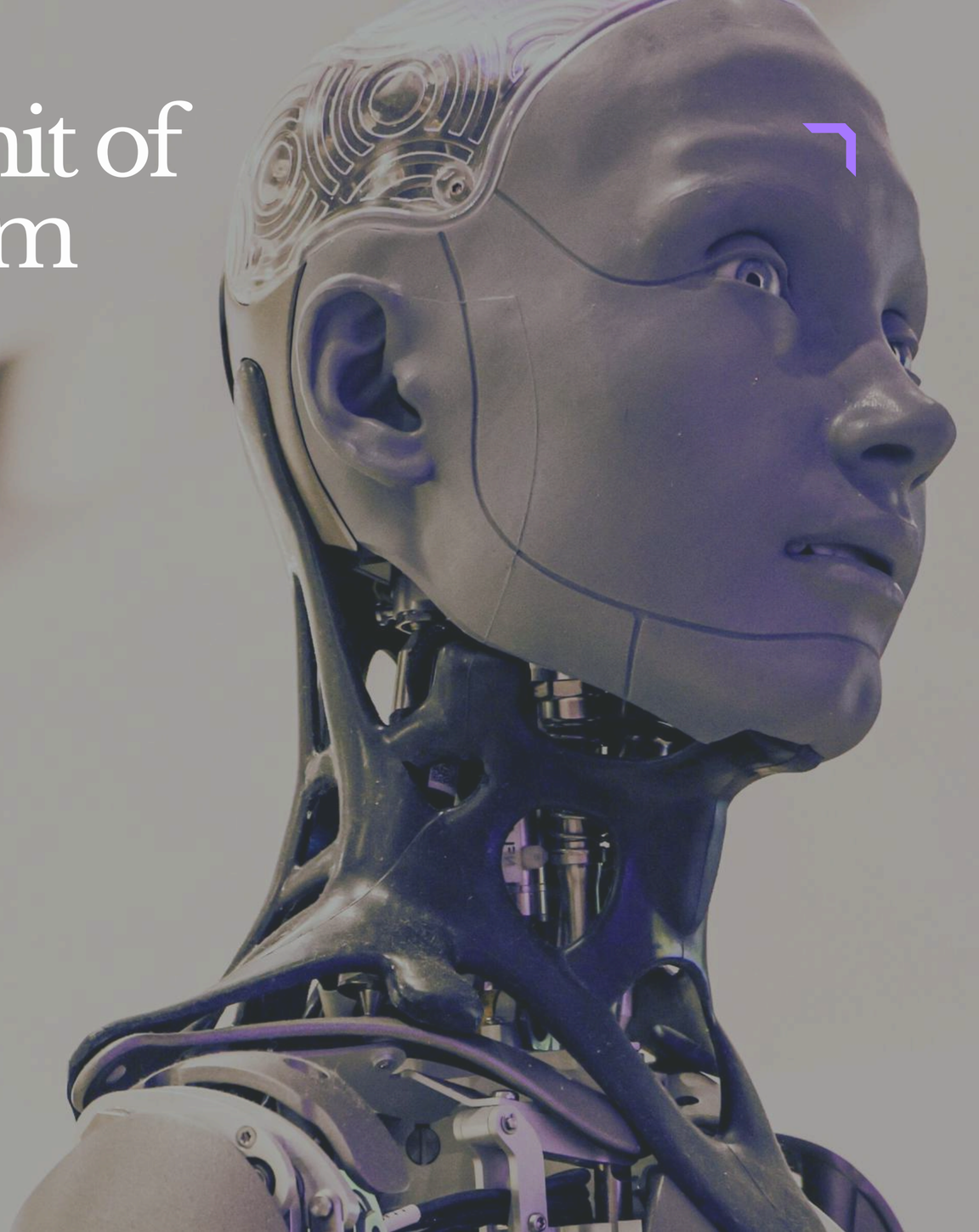


Neuron: Fundamental Unit of Brain & Nervous System

Structure of Neuron

Neuron has 3 main parts:

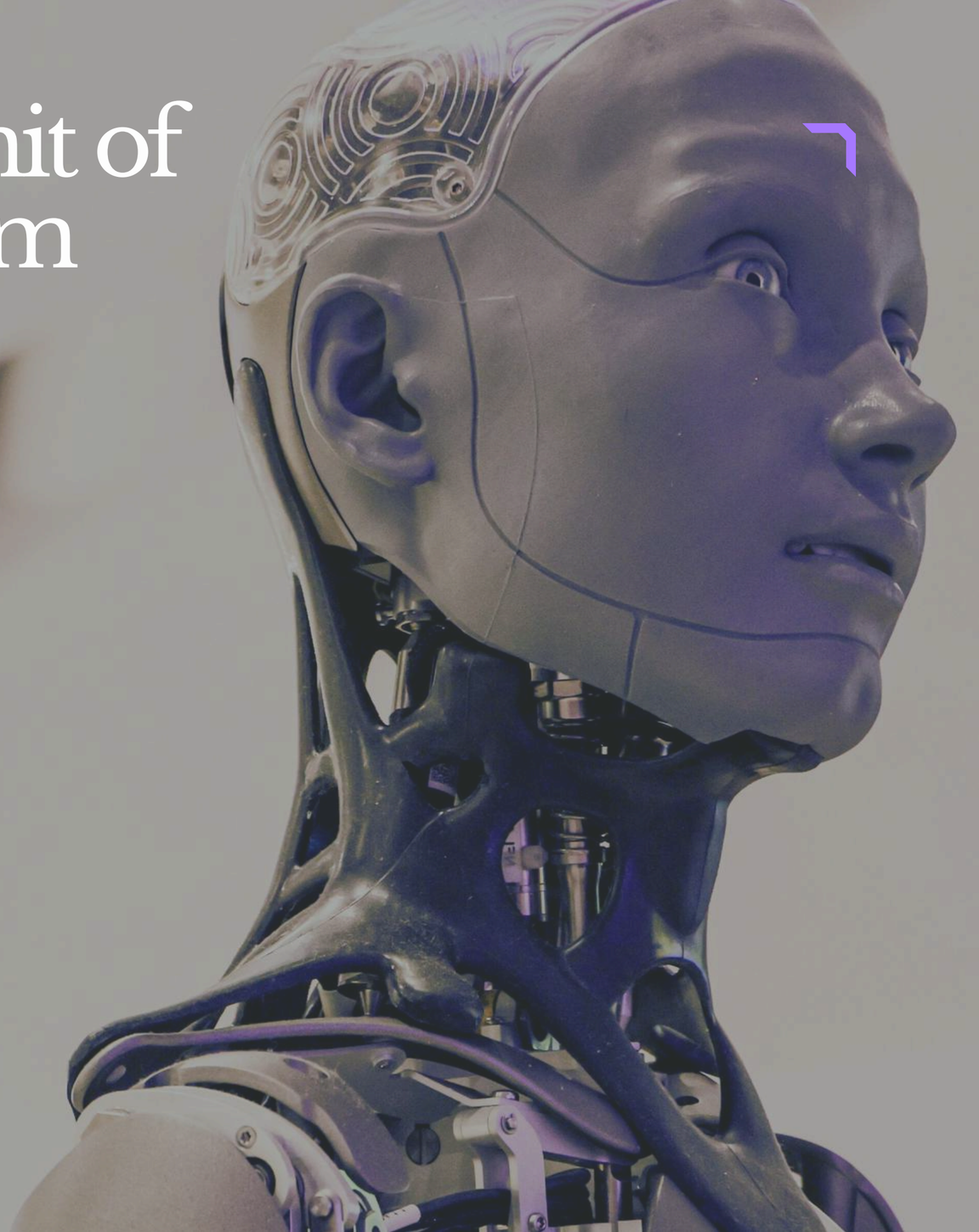
- **Dendrites**
- **Cell Body (Soma)**
- **Axon**



Neuron: Fundamental Unit of Brain & Nervous System

Dendrites

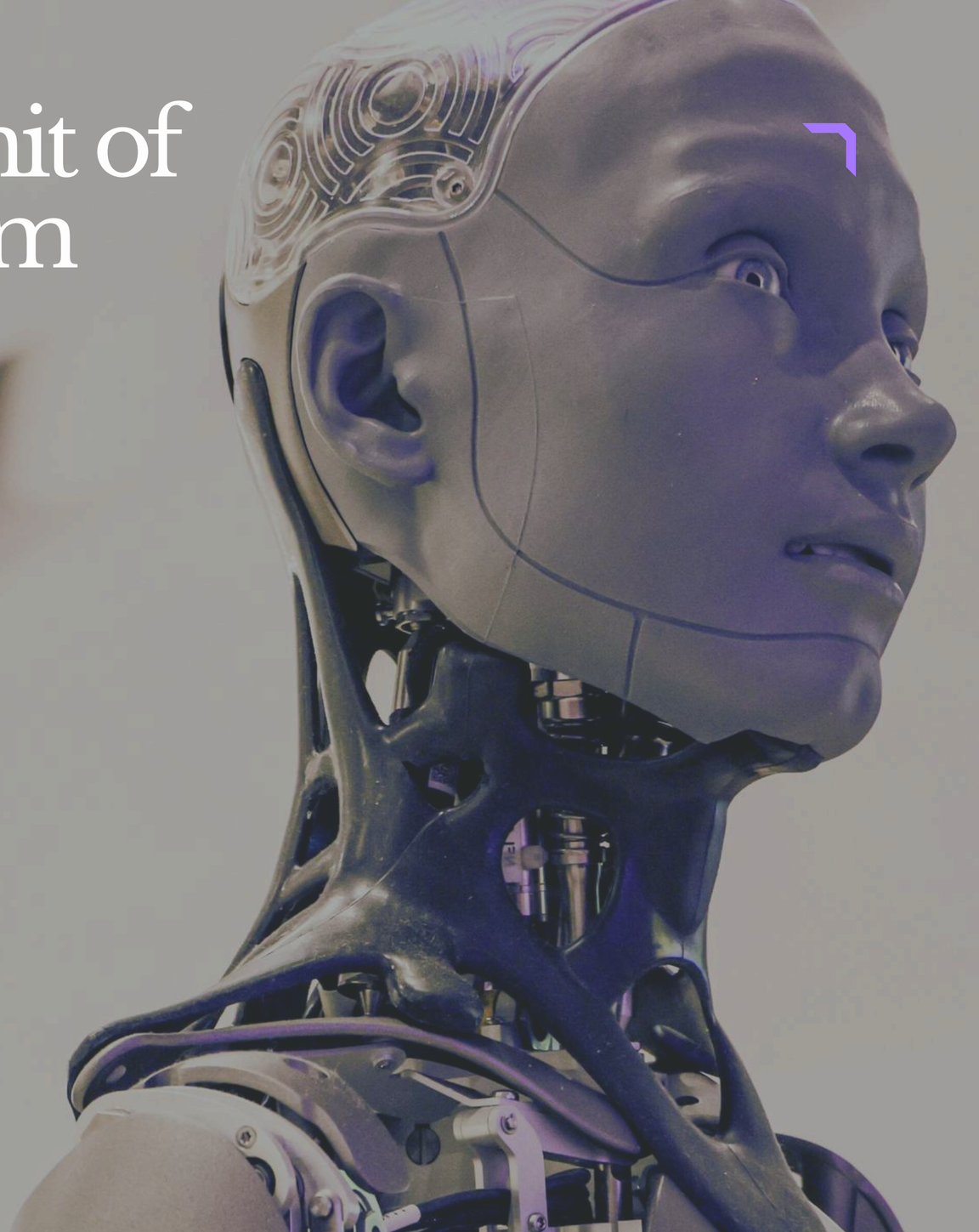
- Branch-like structures.
- Receive signals from other neurons.
- Act like antennas collecting information.



Neuron: Fundamental Unit of Brain & Nervous System

Cell Body (Soma)

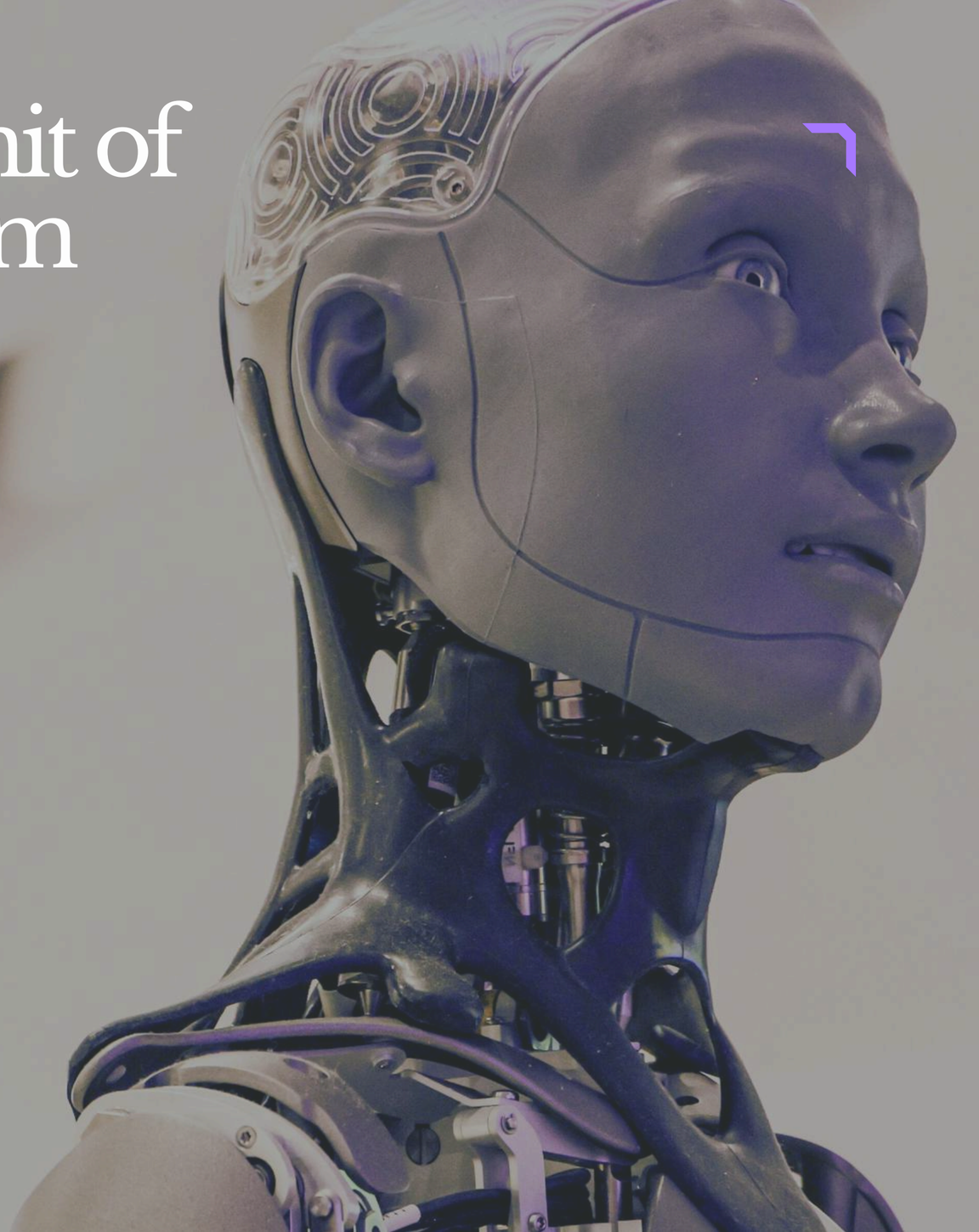
- Processes received signals.
- Decides whether to send a new signal.
- Contains nucleus that controls neuron functions.



Neuron: Fundamental Unit of Brain & Nervous System

Axon

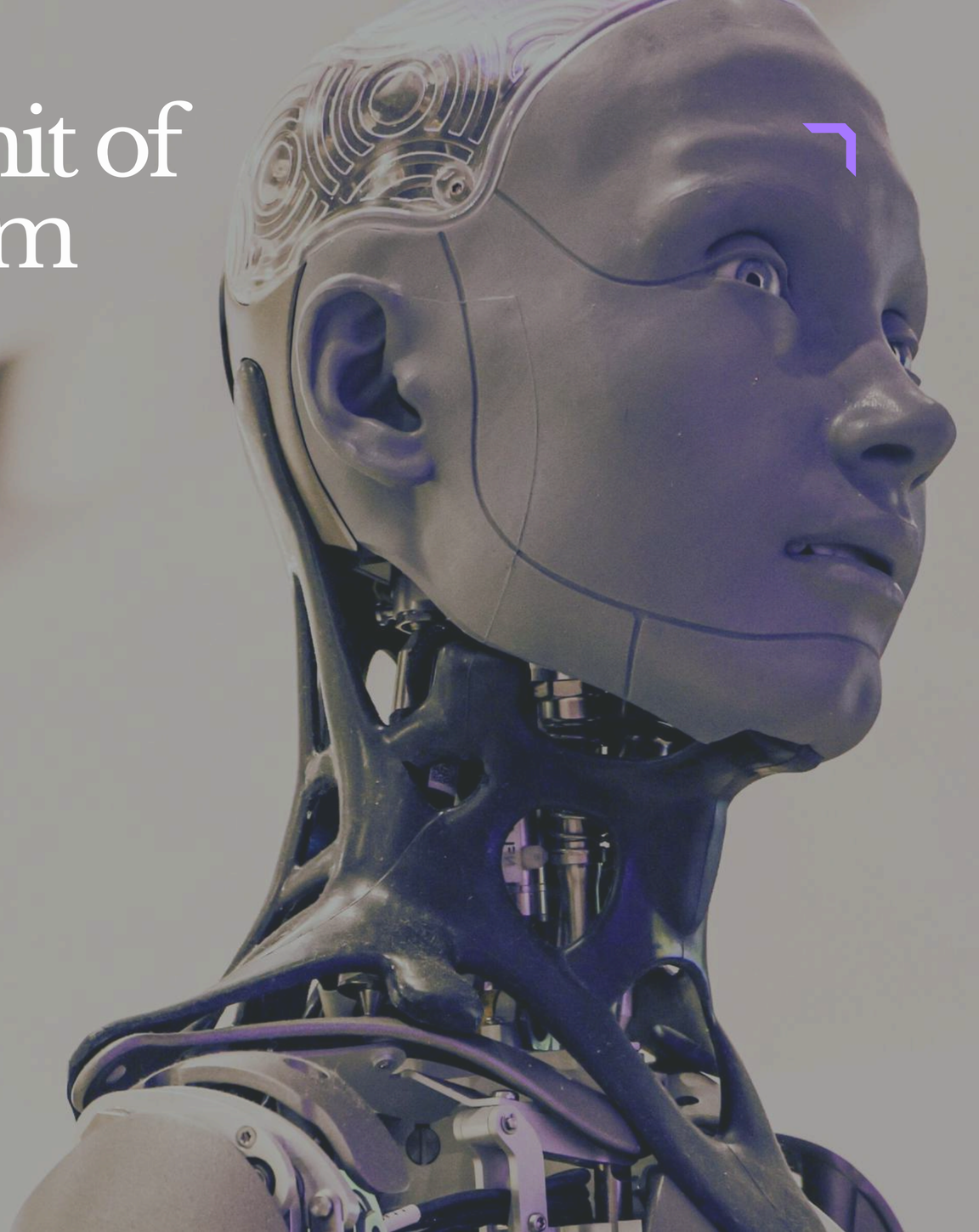
- Long fiber-like structure.
- Carries electrical signals away from cell body.
- Sends signals to other neurons, muscles, or glands.



Neuron: Fundamental Unit of Brain & Nervous System

Synapse & Communication

- Neurons communicate through synapses.
- Electrical signals convert into chemical signals.
- Chemical signals are transmitted using neurotransmitters.



Perceptron

Definition

- A Perceptron is the simplest type of artificial neural network.
- It is inspired by the working of biological neurons in the human brain.
- It is a mathematical model used for classification tasks.
- It takes multiple input values.
- Each input is multiplied by a weight.

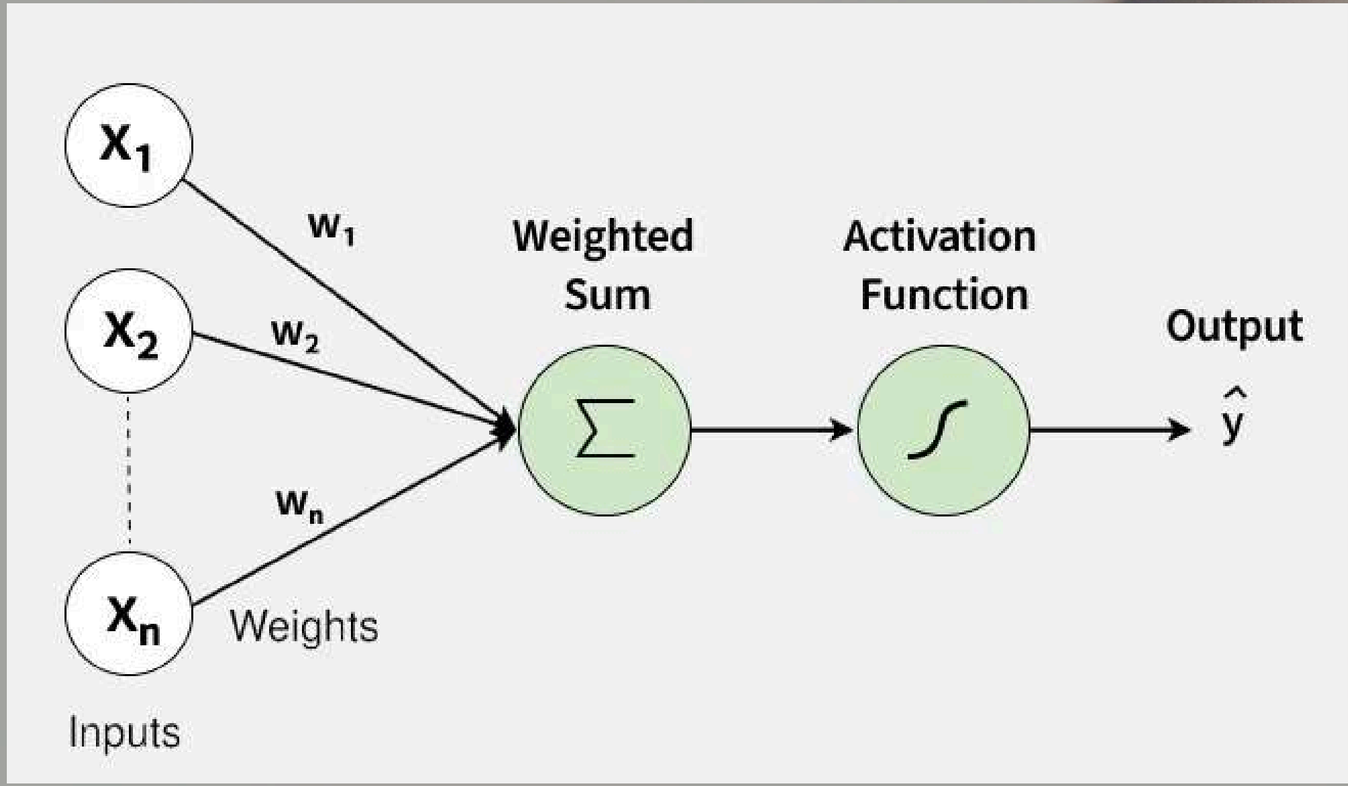
Perceptron

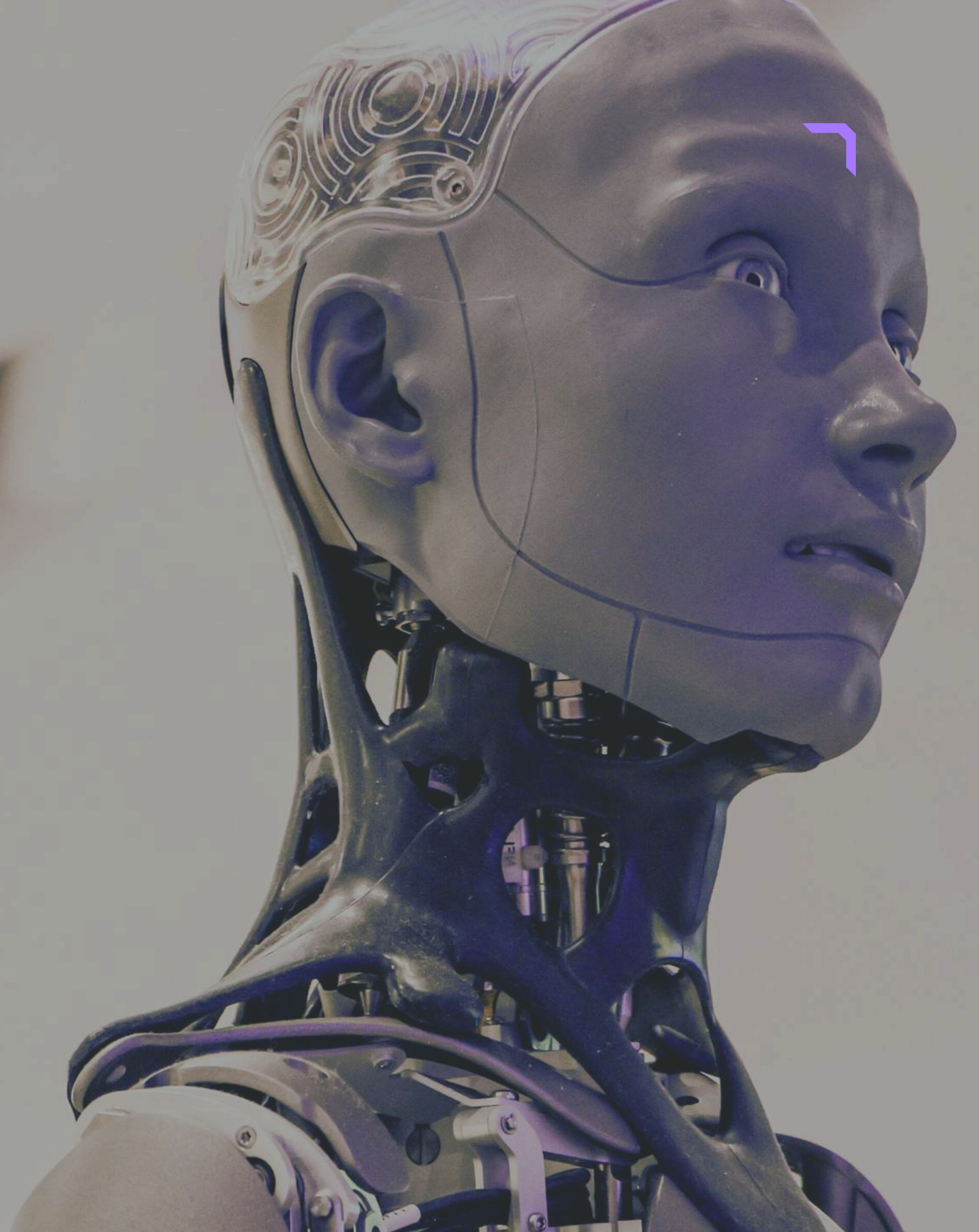
Definition

- All weighted inputs are added together.
- A threshold (activation function) is applied to produce the final output.
- The output is usually binary (either 0 or 1).
- It is mainly used for binary classification problems.

Perceptron

Diagram





Perceptron

Steps for Training a Perceptron

Initialize Weights and Bias

- Assign random values to weights.
- Assign a bias value (a constant added to the summation).

Provide Training Data

- Give input features.
- Provide expected output labels.

Calculate the Weighted Sum

Perceptron

Steps for Training a Perceptron

Multiply each input by its corresponding weight.

- Add all weighted inputs along with bias.
- Formula: z=w1x1+w2x2+...+wnxn+b

Apply Activation Function

- Use a step function (or any activation function).
- Generate the final output (0 or 1).
- y=1 if z>=0 else 0

Check the Output

Perceptron

Steps for Training a Perceptron

- Compare predicted output with actual output.
- If correct → No change needed.
- If incorrect → Update weights and bias.

Repeat the Process

- Repeat steps 2 to 5 for multiple iterations (epochs).
- Continue until the perceptron correctly classifies most of the training data.

Multilayer Feedforward Network (MLFN)

Definition

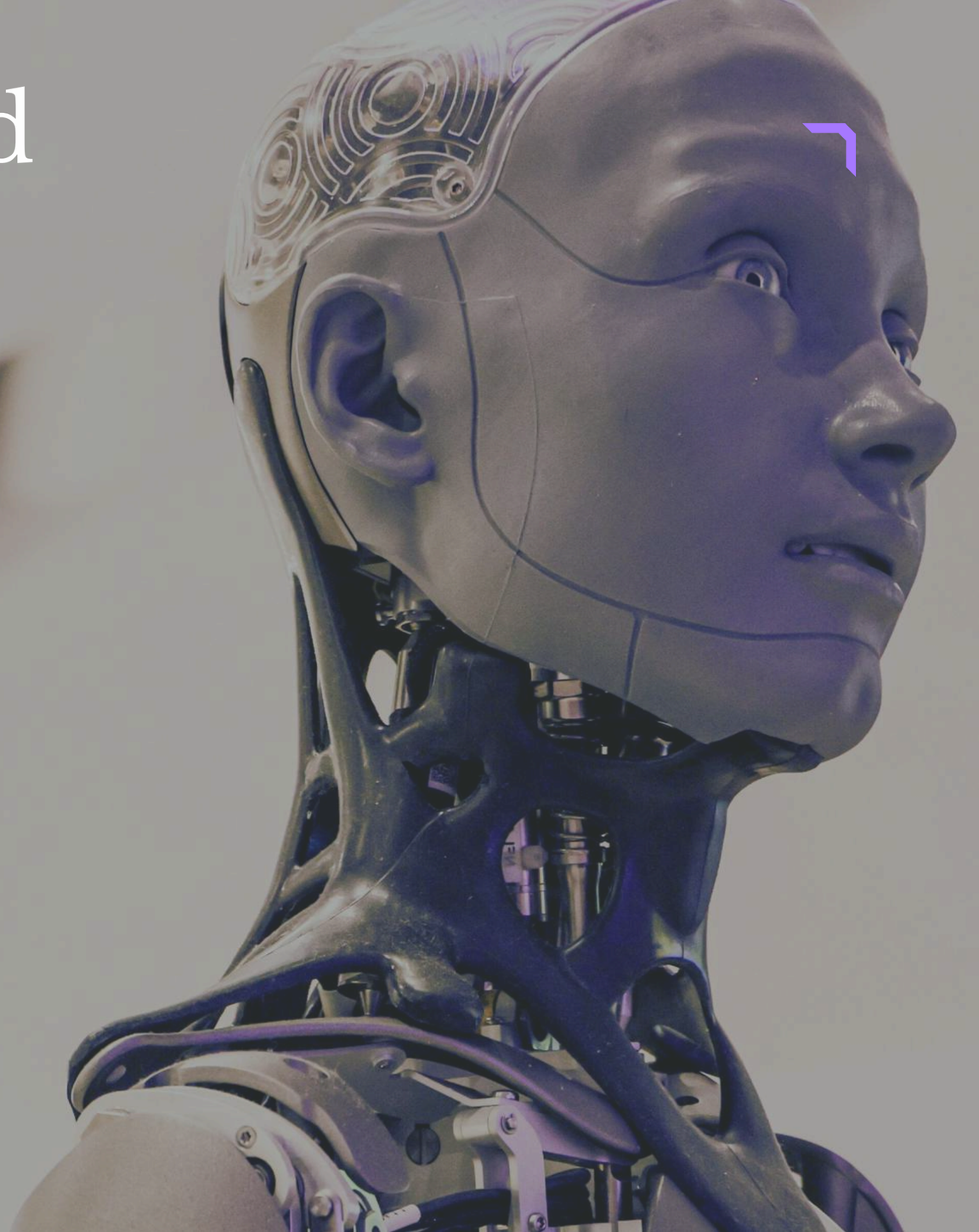
- A Multilayer Feedforward Network (MLFN) is an advanced type of Artificial Neural Network.
- It consists of multiple layers of neurons.
- It allows the network to learn complex patterns from data.
- Information flows only in one direction — from input layer to output layer.
- There is no feedback connection (no loops).

Multilayer Feedforward Network (MLFN)

Definition

Layers in MLFN

- **Input Layer – Receives input features.**
- **Hidden Layer(s) – Processes the inputs using weights and activation functions.**
- **Output Layer – Produces the final result.**



Multilayer Feedforward Network (MLFN)

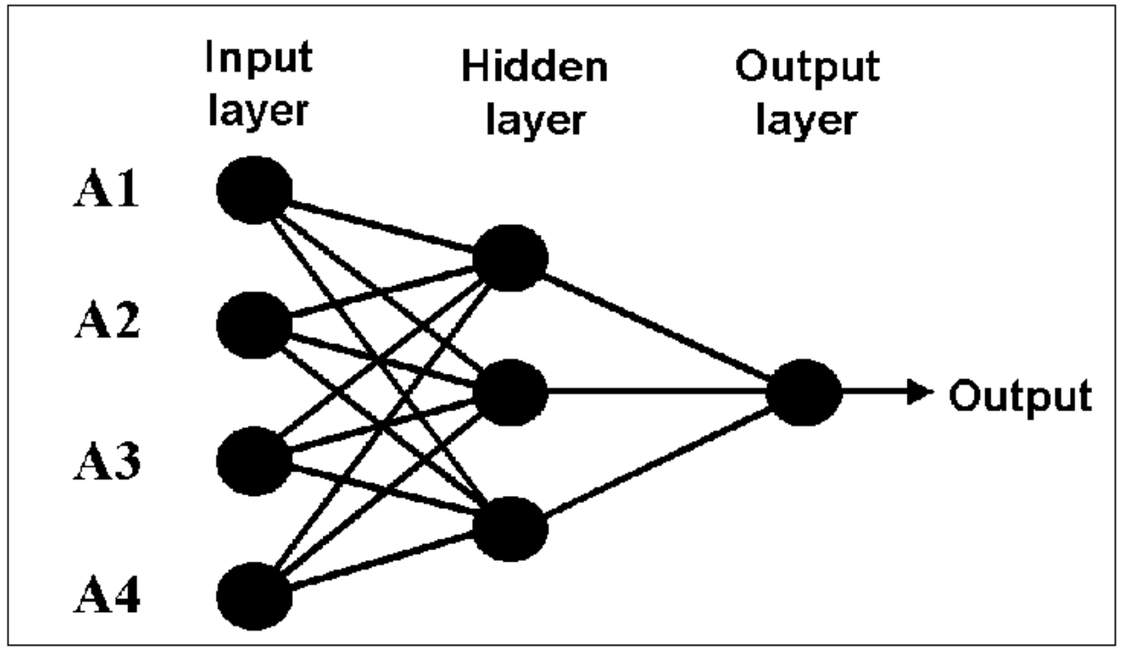
Definition

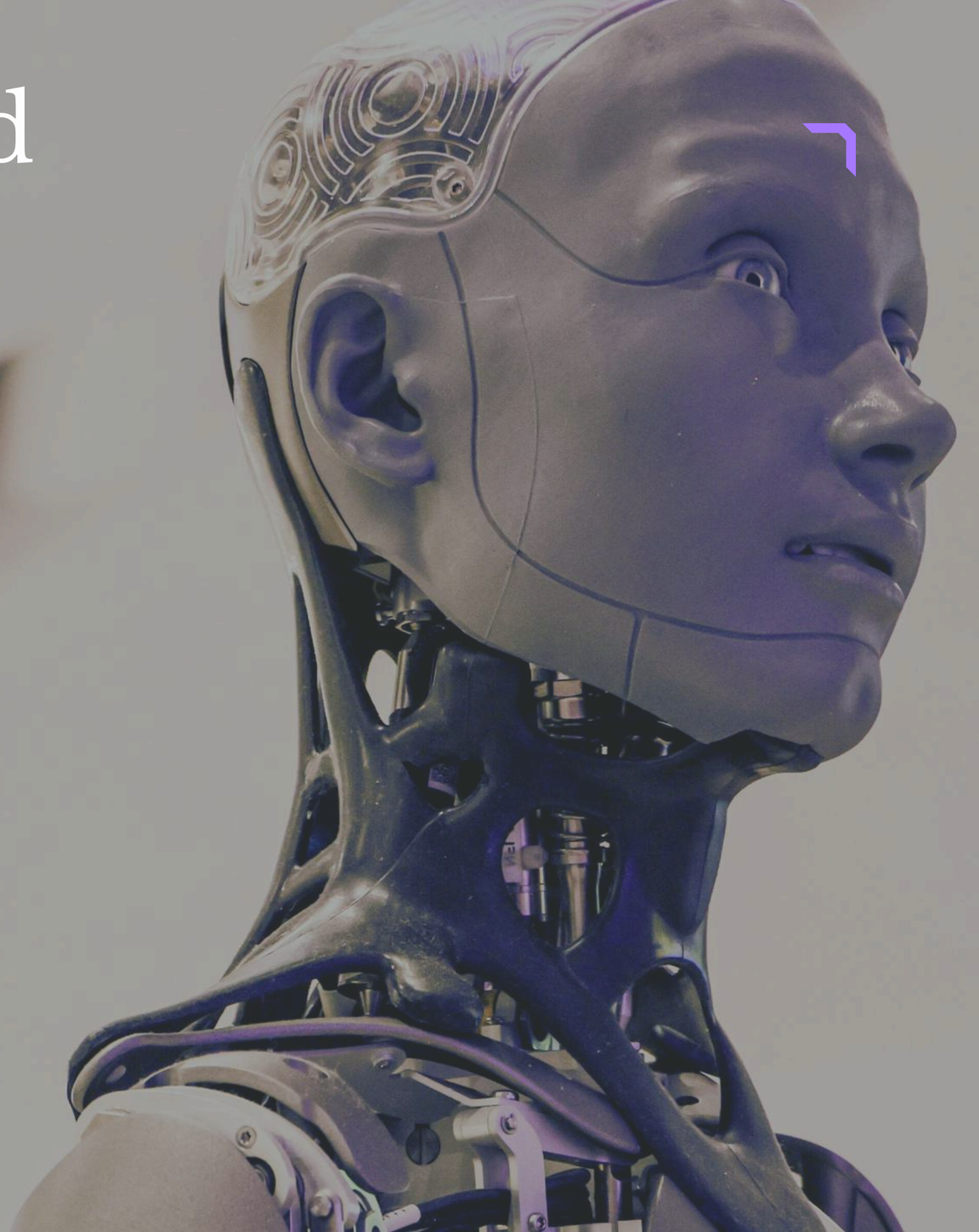
Difference from Perceptron

- A simple perceptron has only one layer.
- It can solve only simple linear problems.
- A multilayer network has multiple layers.
- It is more powerful and can solve complex (non-linear) problems.

Multilayer Feedforward Network (MLFN)

Diagram



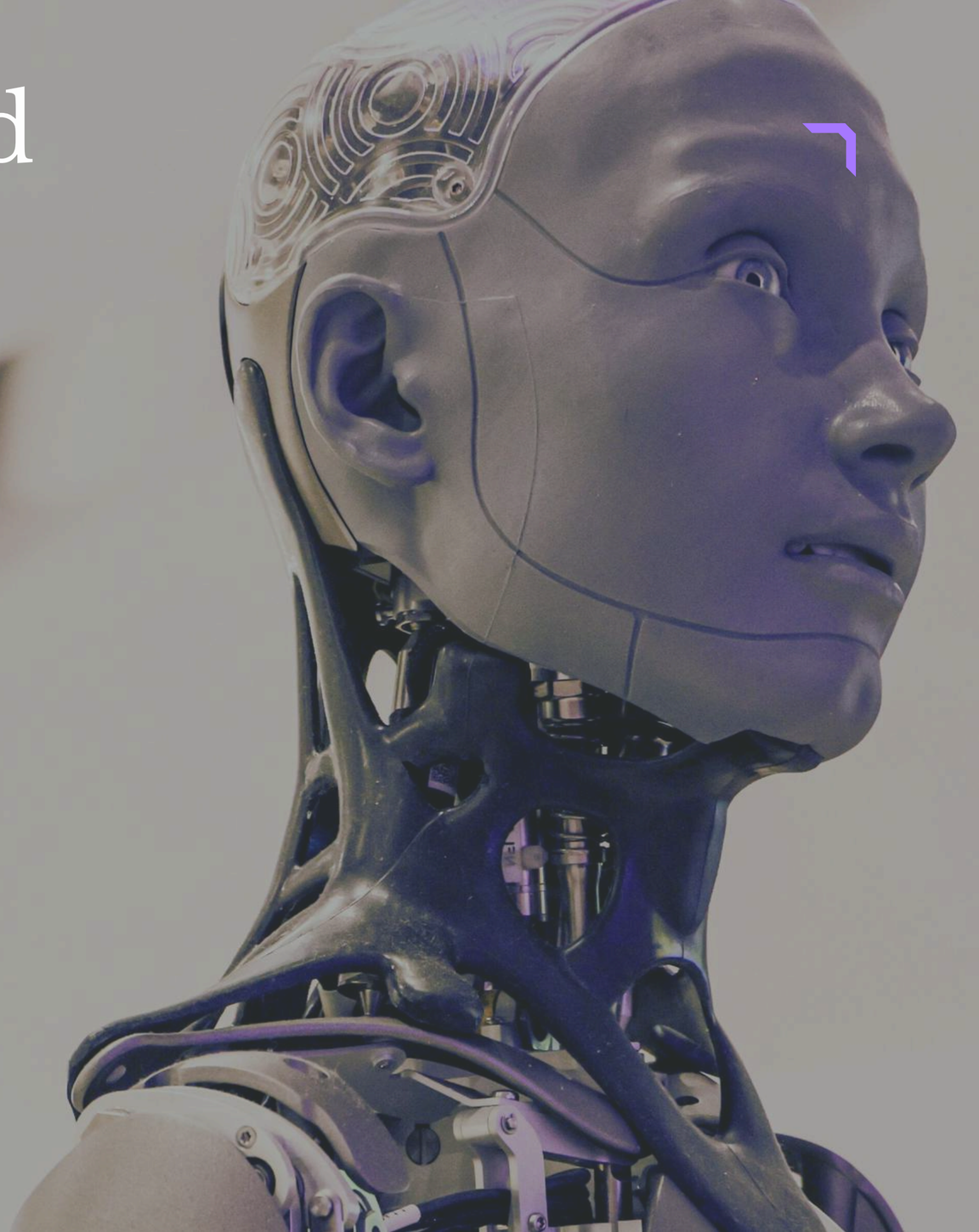


Multilayer Feedforward Network (MLFN)

Main Types of Layers in MLFN

1. Input Layer
   - This layer receives raw data (input features).
   - It passes the data to the next layer.

2. Hidden Layer(s)
   - These layers process the input data.
   - Each neuron applies weights and an activation function.
   - One or more hidden layers can be present.

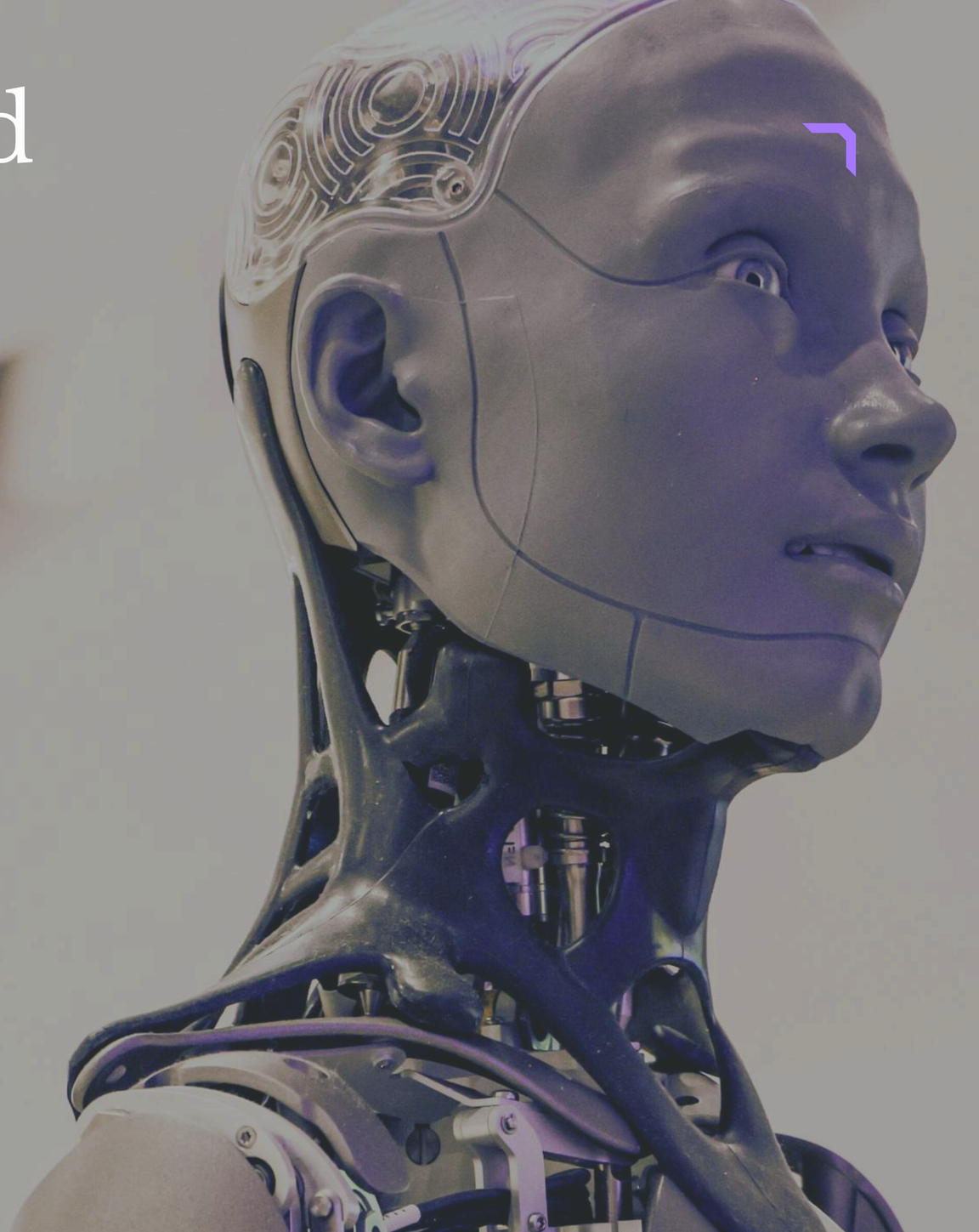


Multilayer Feedforward Network (MLFN)

Main Types of Layers in MLFN

3. Output Layer

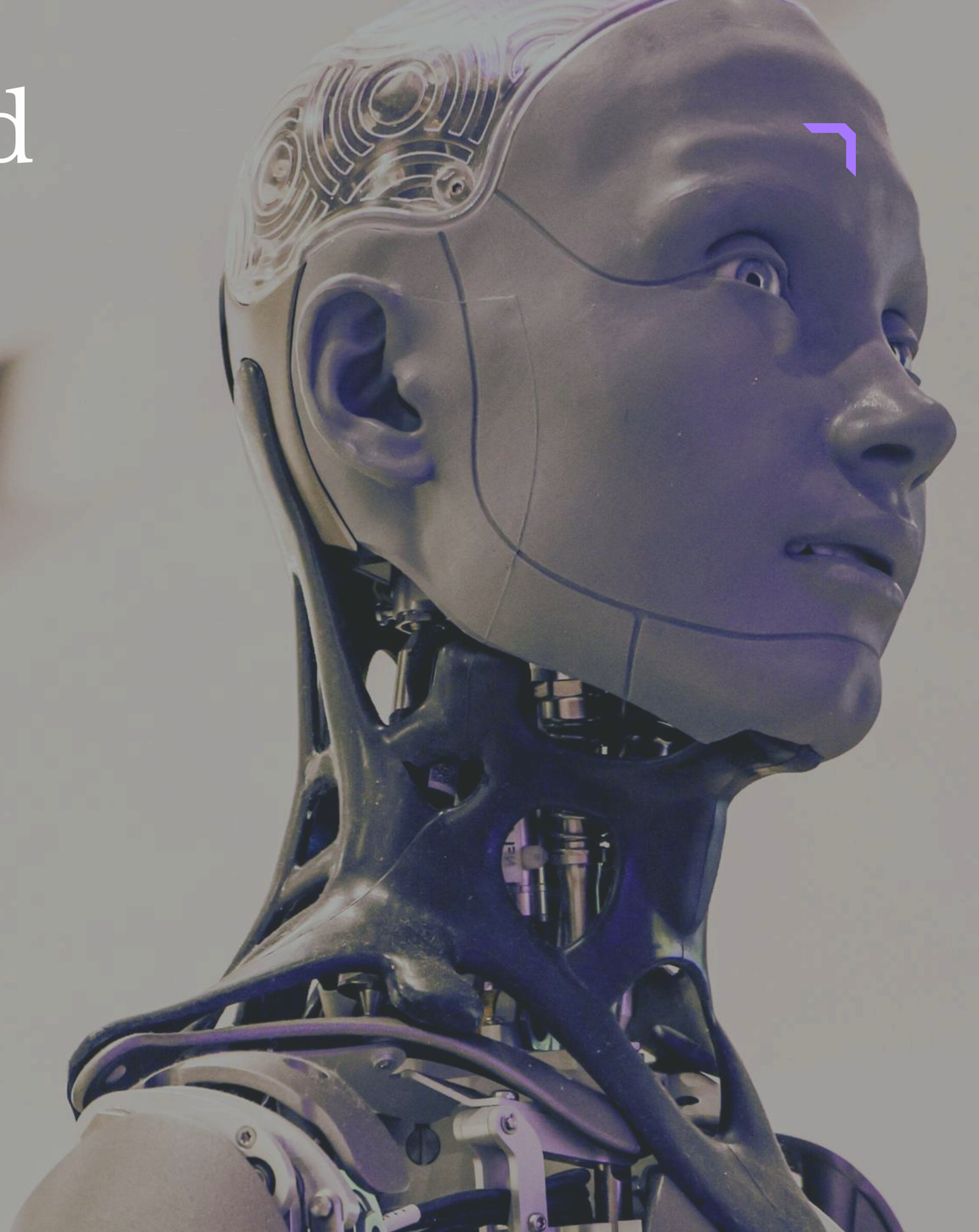
- **This layer produces the final prediction or classification result.**
- **The output is based on the processed information from hidden layers.**



Multilayer Feedforward Network (MLFN)

working of mlfn

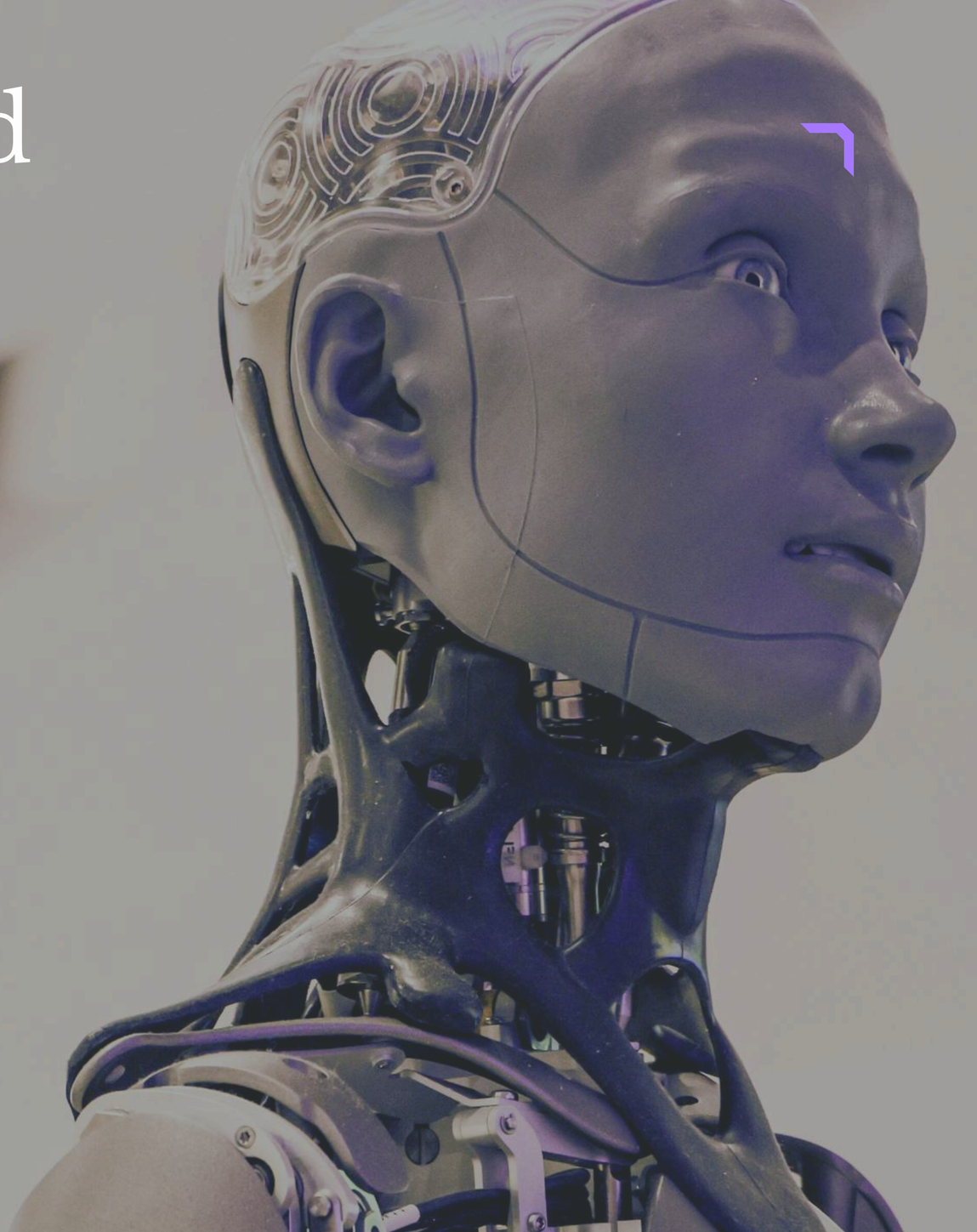
- Each neuron in one layer is connected to every neuron in the next layer (fully connected network).
- Information flows only in the forward direction (from input to output).
- There are no loops or feedback connections.
- That is why it is called a Feedforward network.



Multilayer Feedforward Network (MLFN)

Training Process

- These networks are trained using a process called Backpropagation.
- Backpropagation adjusts the weights of the network.
- The goal is to minimize errors and improve accuracy.



Forward Propagation and Backpropagation

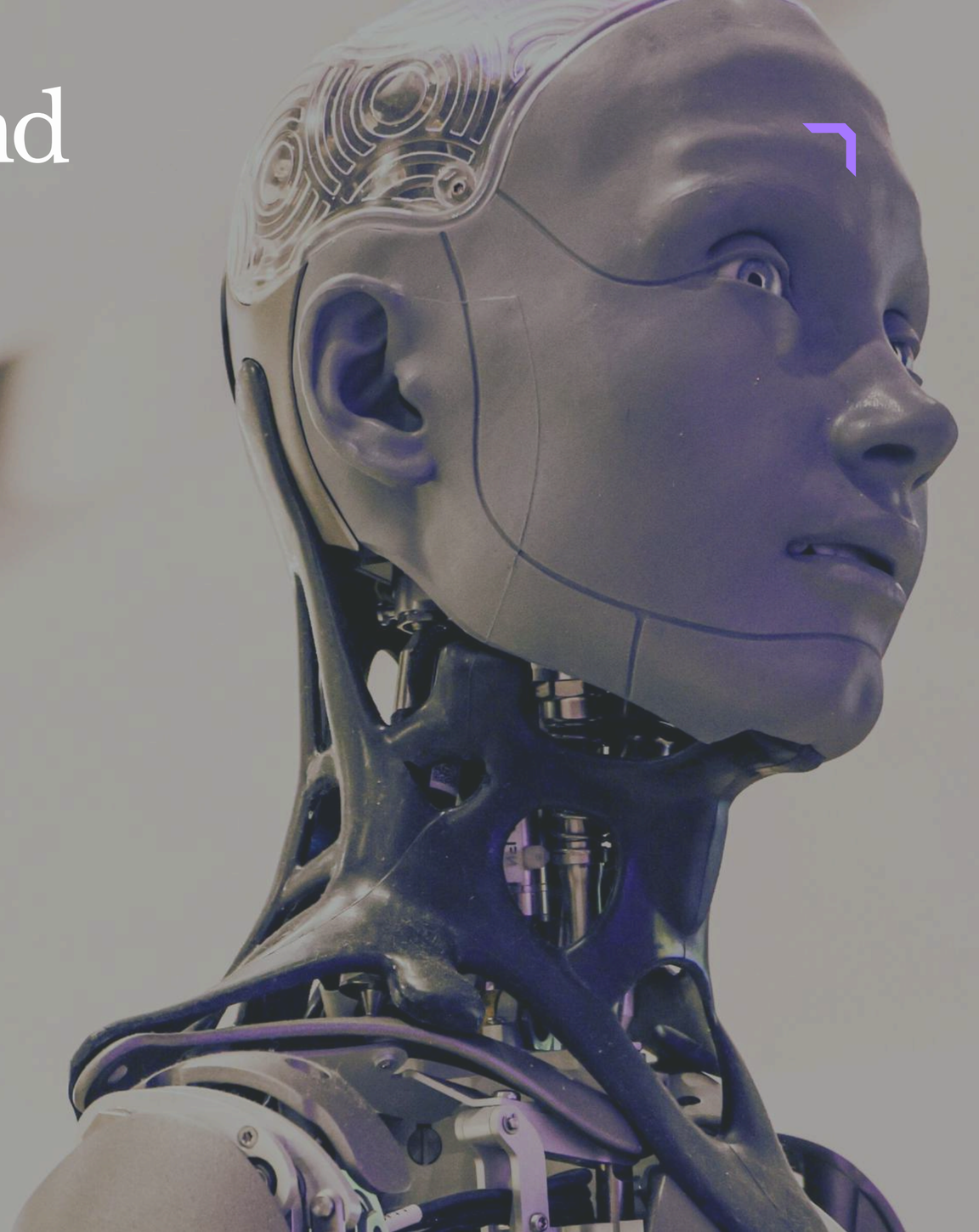
Forward Propagation

- Forward propagation is the process where input data moves forward through the network layer by layer.
- It generates the final output (prediction).

Steps in Forward Propagation

a.Input Layer

- Input values are fed into the neural network.



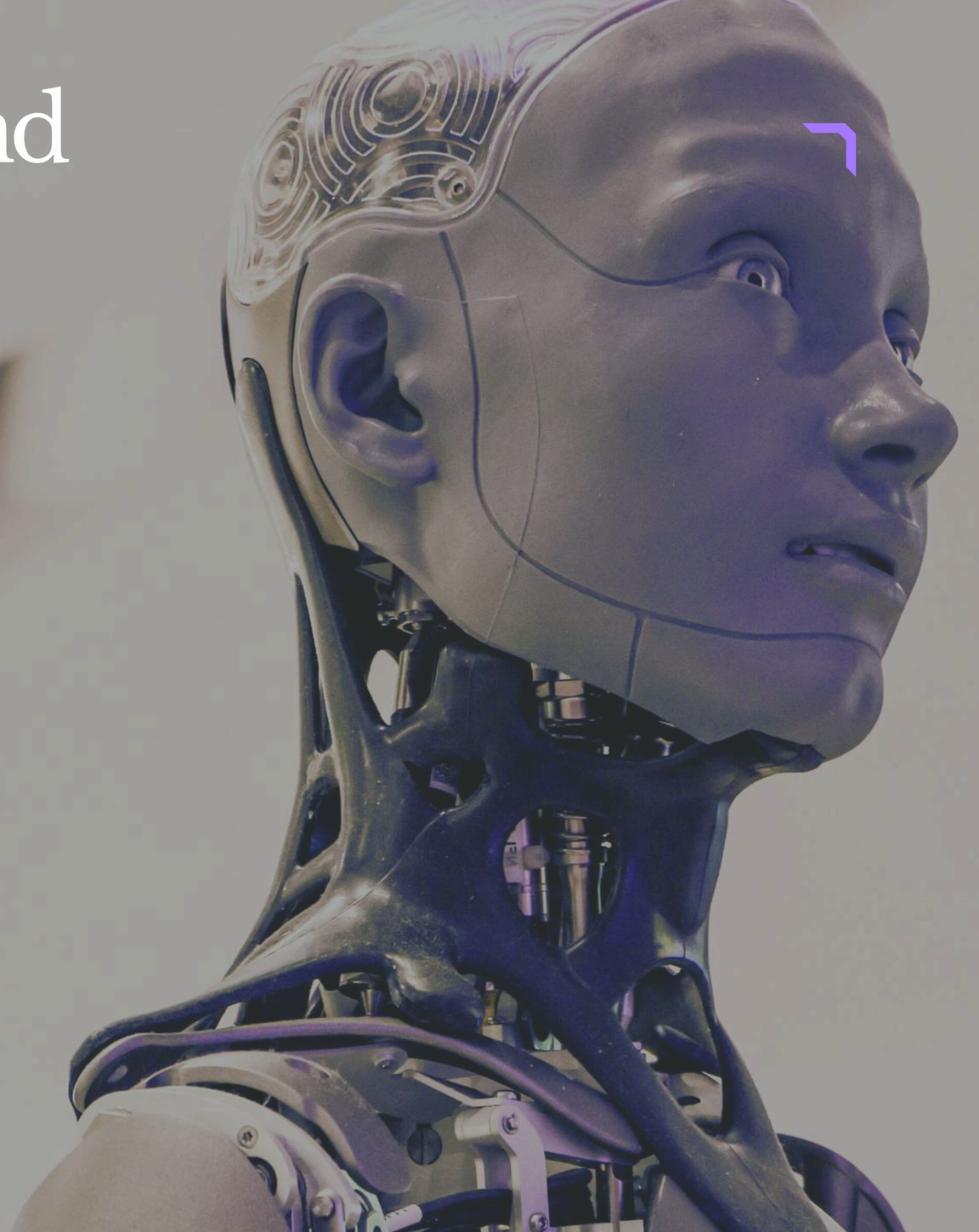
Forward Propagation and Backpropagation

Forward Propagation

- Weighted Sum Calculation
- Each neuron calculates the weighted sum of inputs.
- Formula: z=w1x1+w2x2+...+wnxn+b

b.Apply Activation Function

- An activation function (e.g., ReLU, Sigmoid) is applied.
- It introduces non-linearity and decides



Forward Propagation and Backpropagation

Forward Propagation

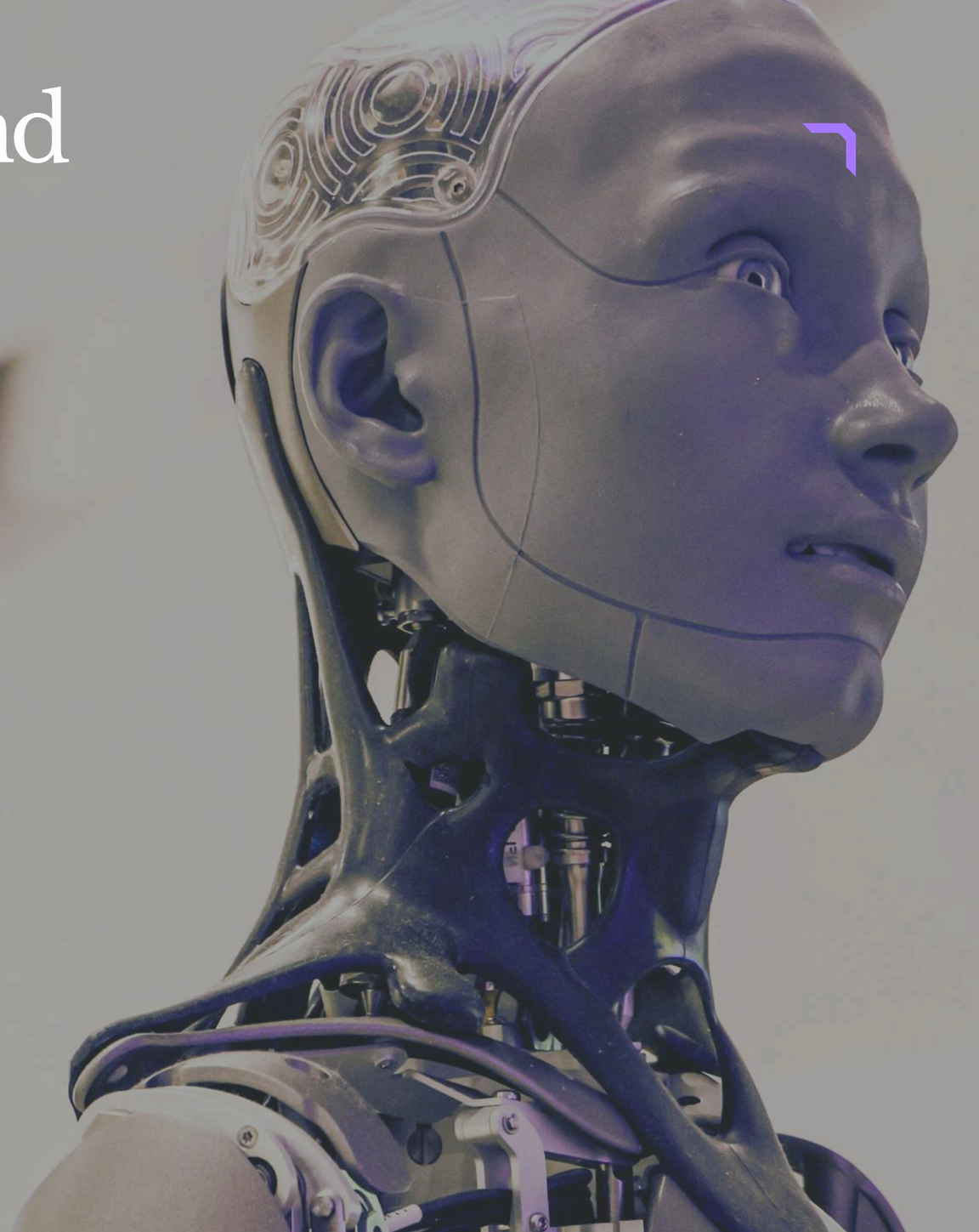
neuron activation.

c.Pass to Next Layer

- The output of one layer becomes the input of the next layer.
- The process repeats layer by layer.

d.Final Output

- The last layer generates the network's prediction.



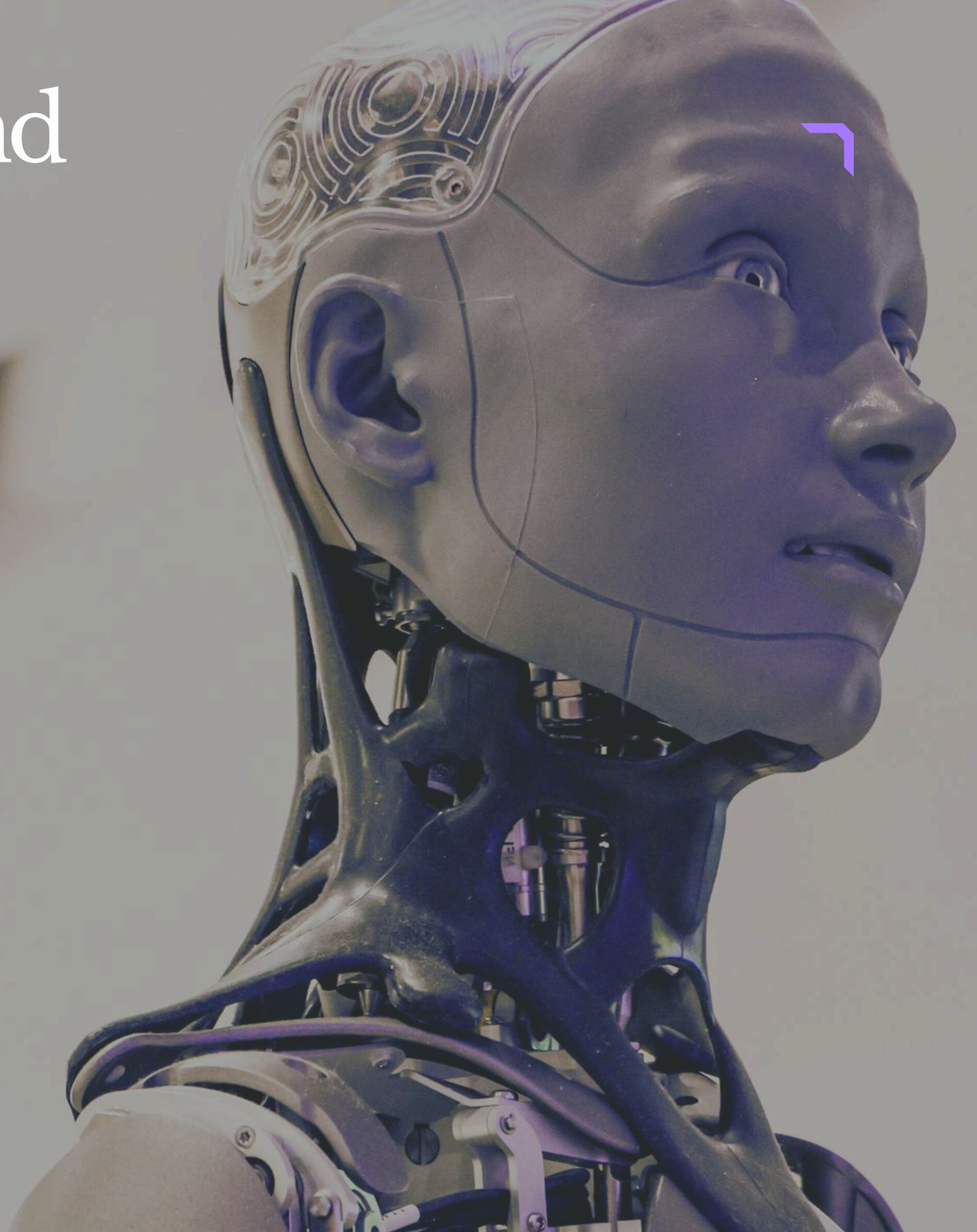
Forward Propagation and Backpropagation

Backward Propagation

- Backpropagation is the process by which the network learns.
- It adjusts the weights based on the error in prediction.
- The error is sent backward through the network to reduce mistakes.

Steps in Backpropagation

1. Calculate Error



Forward Propagation and Backpropagation

Backward Propagation

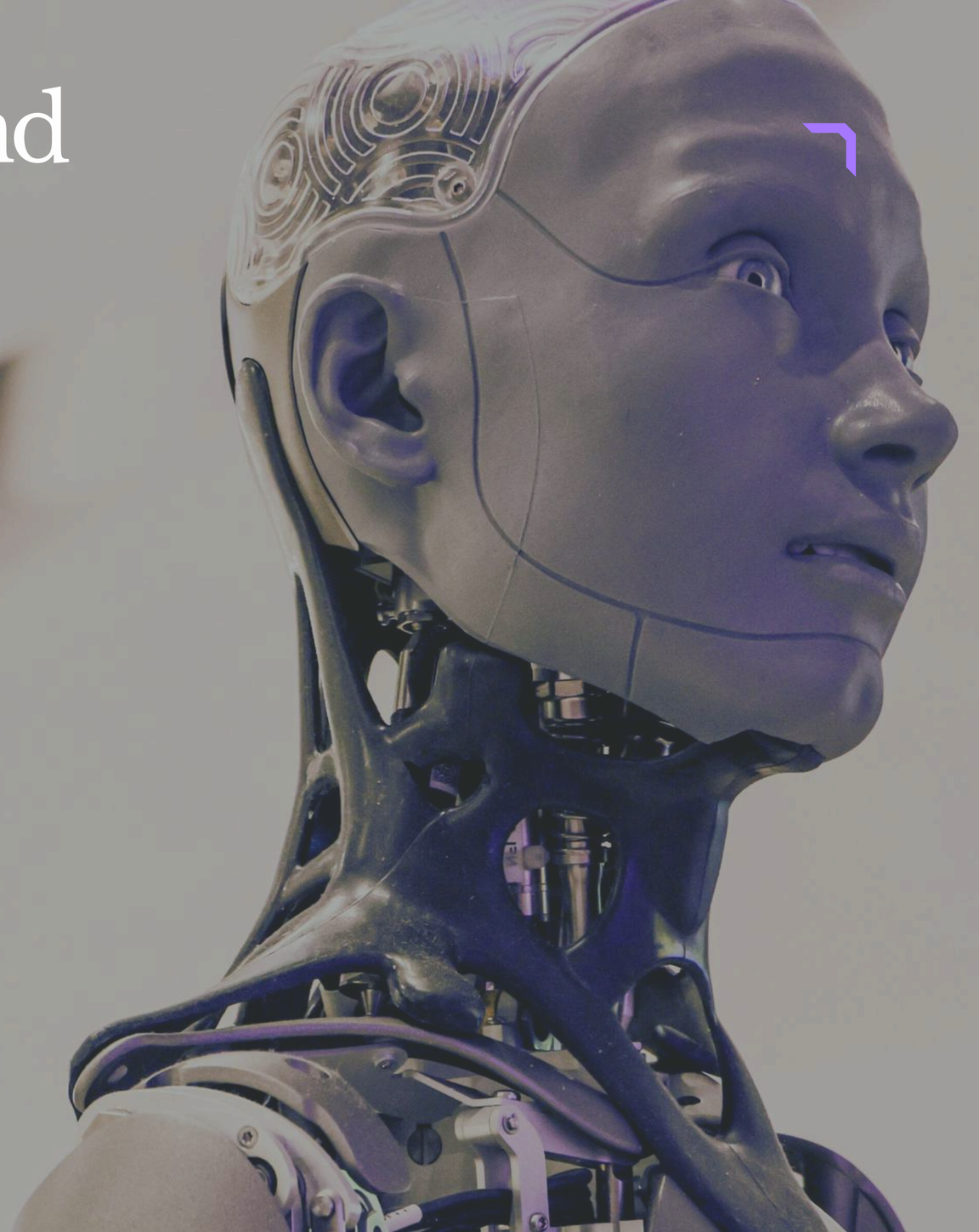
- Compare predicted output with actual output.
- Use a loss function to measure error.

2.Compute Gradients

- Use derivatives to find how much each weight contributed to the error.

3.Update Weights (Gradient Descent)

- Adjust weights using the formula:



Forward Propagation and Backpropagation

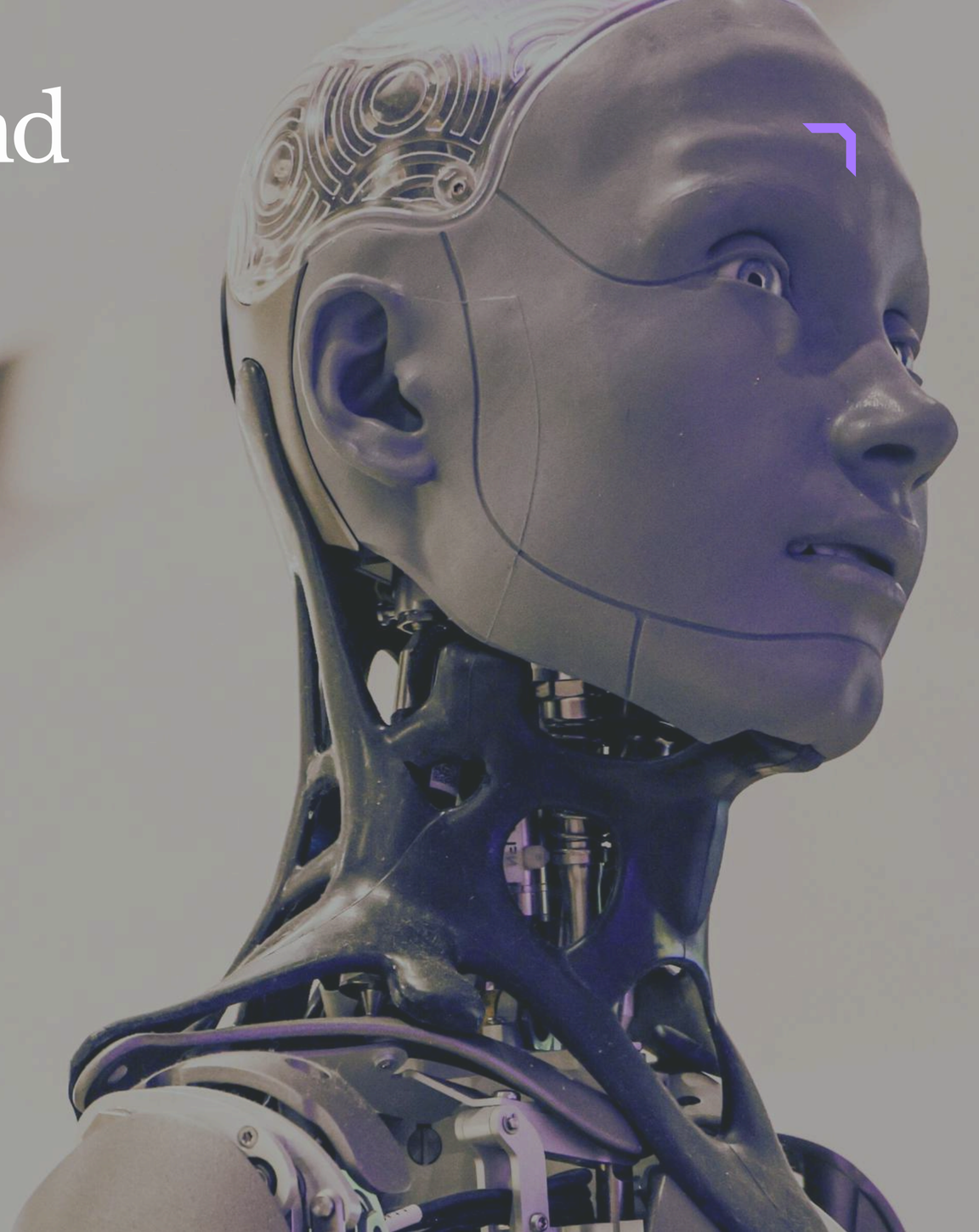
Backward Propagation

$$w_{new} = w_{old} - \eta \frac{\partial Loss}{\partial w}$$

- **Here, η (eta) is the learning rate.**

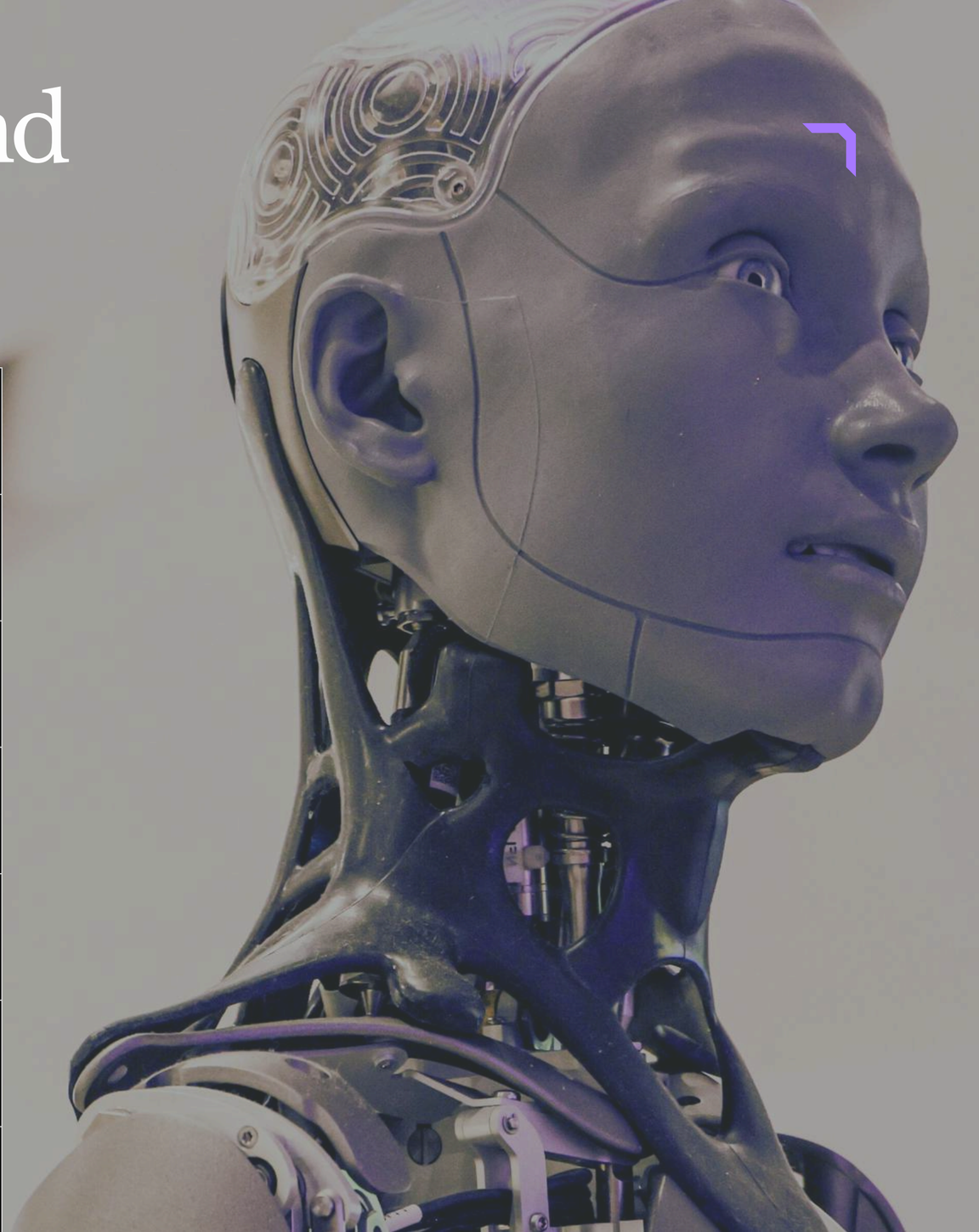
4.Repeat the Process

- **Continue forward and backward propagation.**
- **Stop when error is minimized and accuracy improves.**



Forward Propagation and Backpropagation(diff)

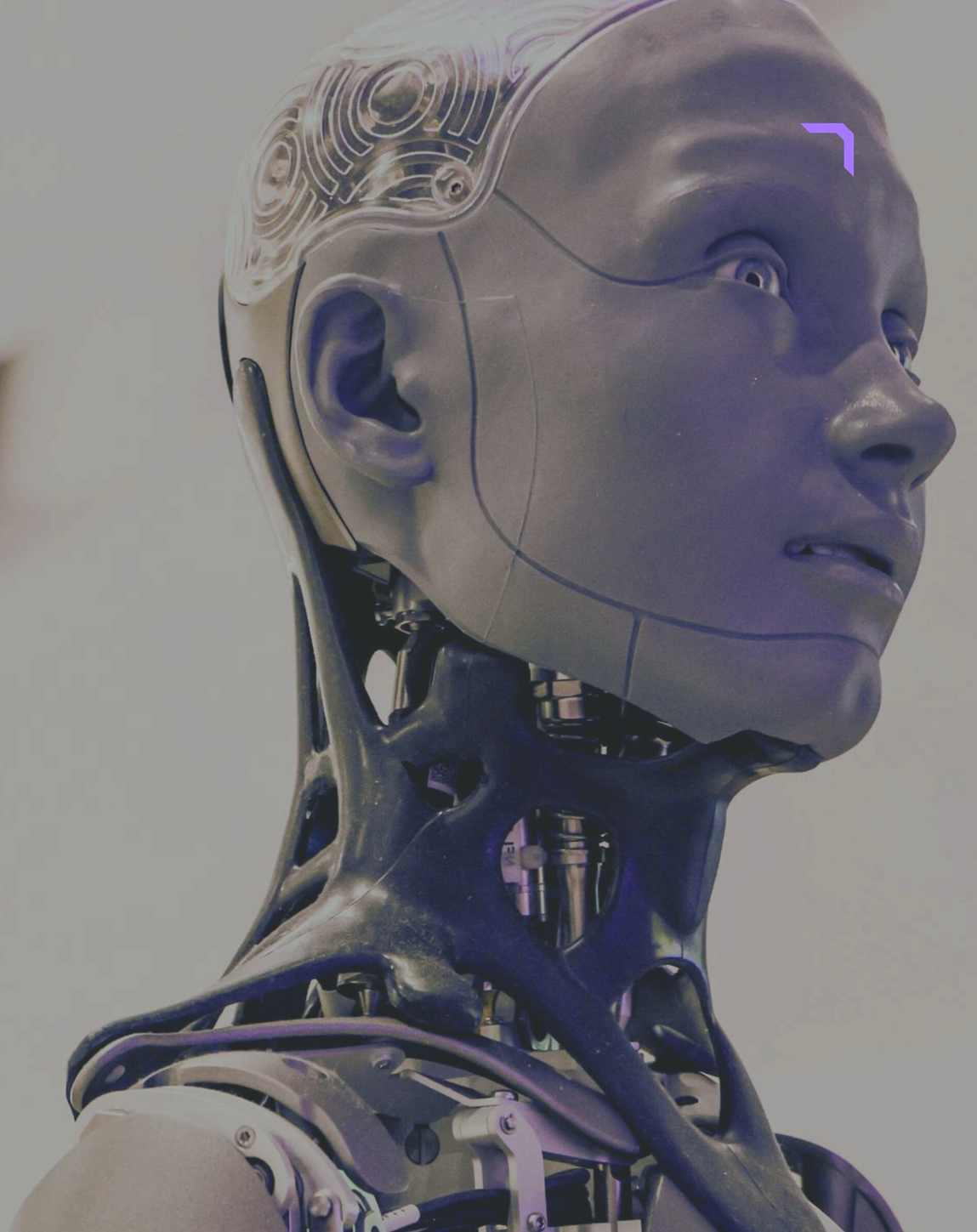
Forward Propagation	Backpropagation
Data moves from input layer to output layer.	Error moves from output layer back to input layer.
Used to generate prediction/output.	Used to reduce error and improve accuracy.
Calculates weighted sum and applies activation function.	Calculates gradients using derivatives.
Does not update weights.	Updates weights using gradient descent.
Happens first during training.	Happens after forward propagation.
Focuses on computing output.	Focuses on minimizing loss (error).



Activation Functions

Definition

- Activation functions help a neural network decide whether a neuron should be activated or not.
- They determine the output of a neuron.
- They introduce non-linearity in the network.
- Non-linearity helps the network learn complex patterns from data.
- Without activation functions, a neural network behaves like a simple linear model.

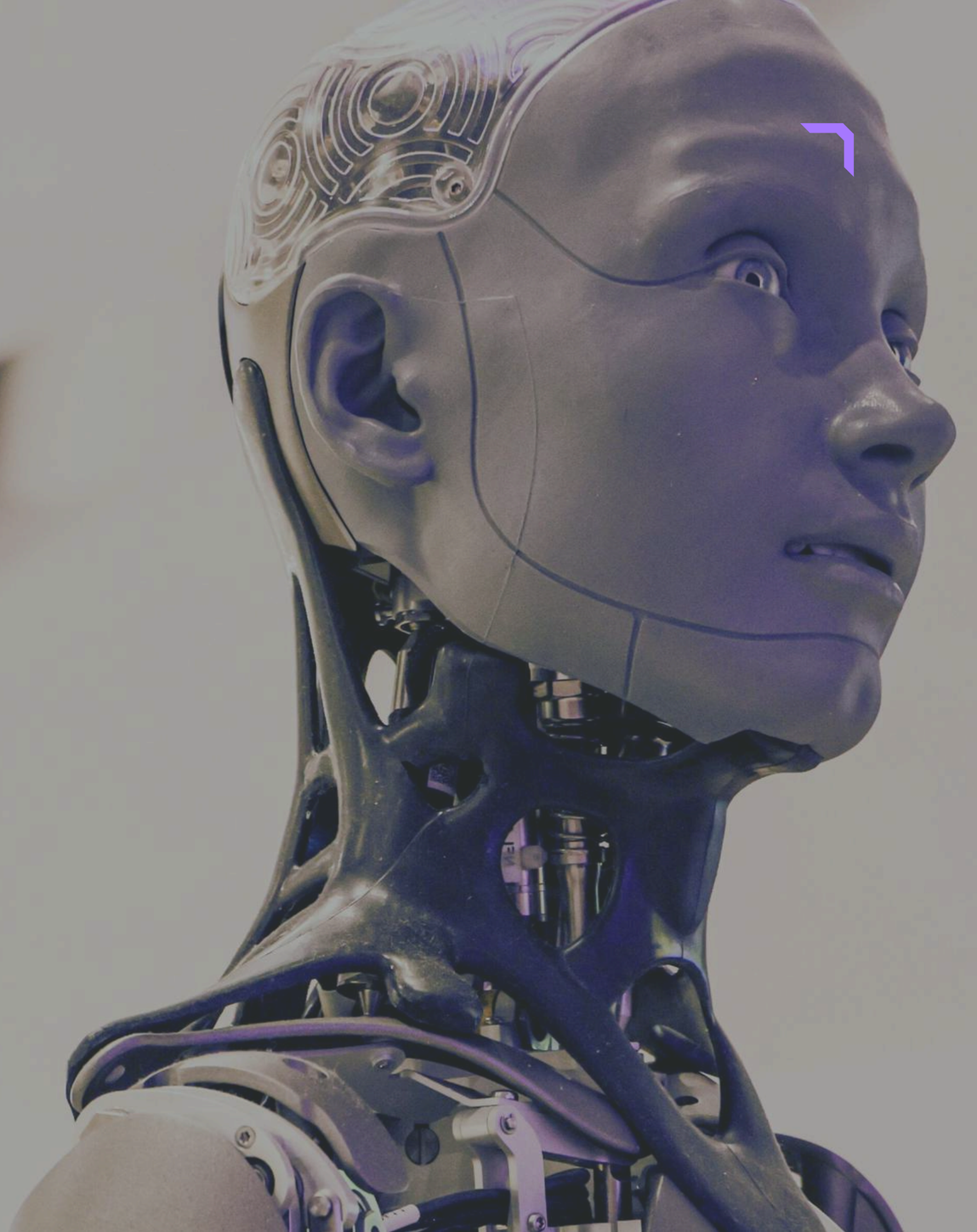


Activation Functions

Definition

Common Activation Functions

- Sigmoid
- ReLU (Rectified Linear Unit)
- Leaky ReLU
- Tanh
- Softmax
- ELU



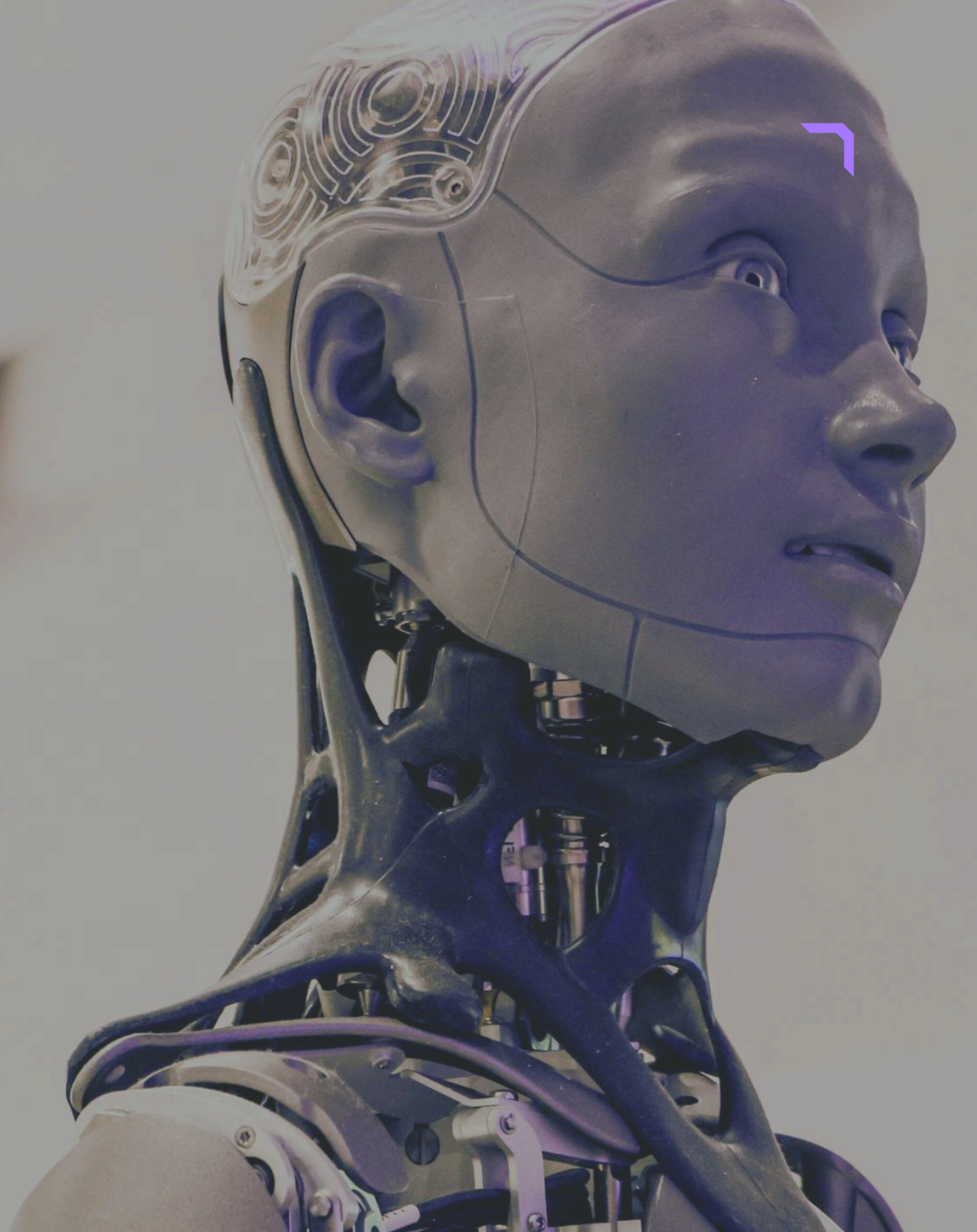
Activation Functions

Sigmoid Activation Function

- The Sigmoid function maps input values between 0 and 1.
- It is mainly used for binary classification problems.
- It converts outputs into probability values.

Formula

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Activation Functions

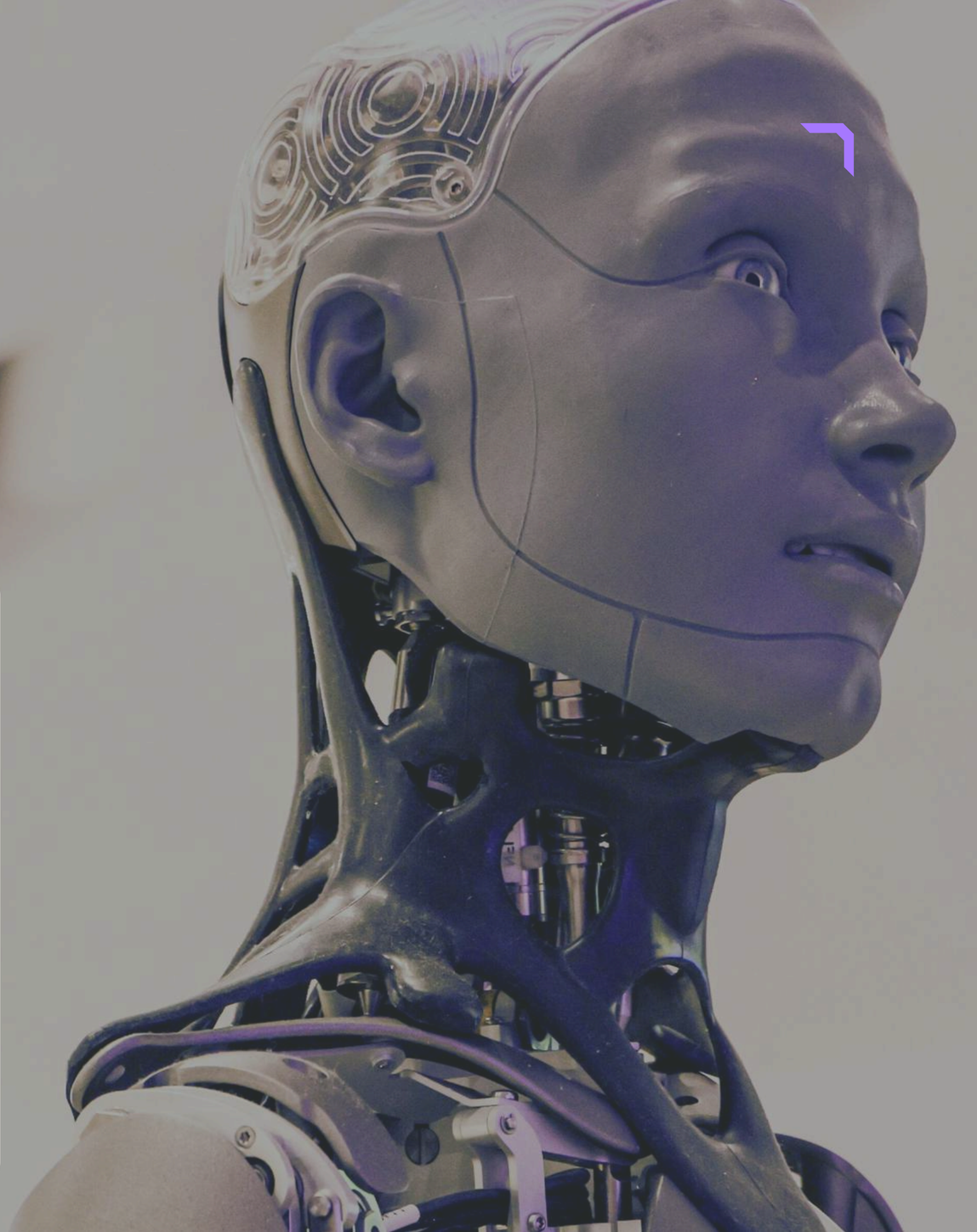
Sigmoid Activation Function

Example

- If x=2
- f(2)≈0.88
- This means the neuron is highly activated.

Use Case

- Used in probability-based predictions
- Example: Spam vs Not Spam email classification



Activation Functions

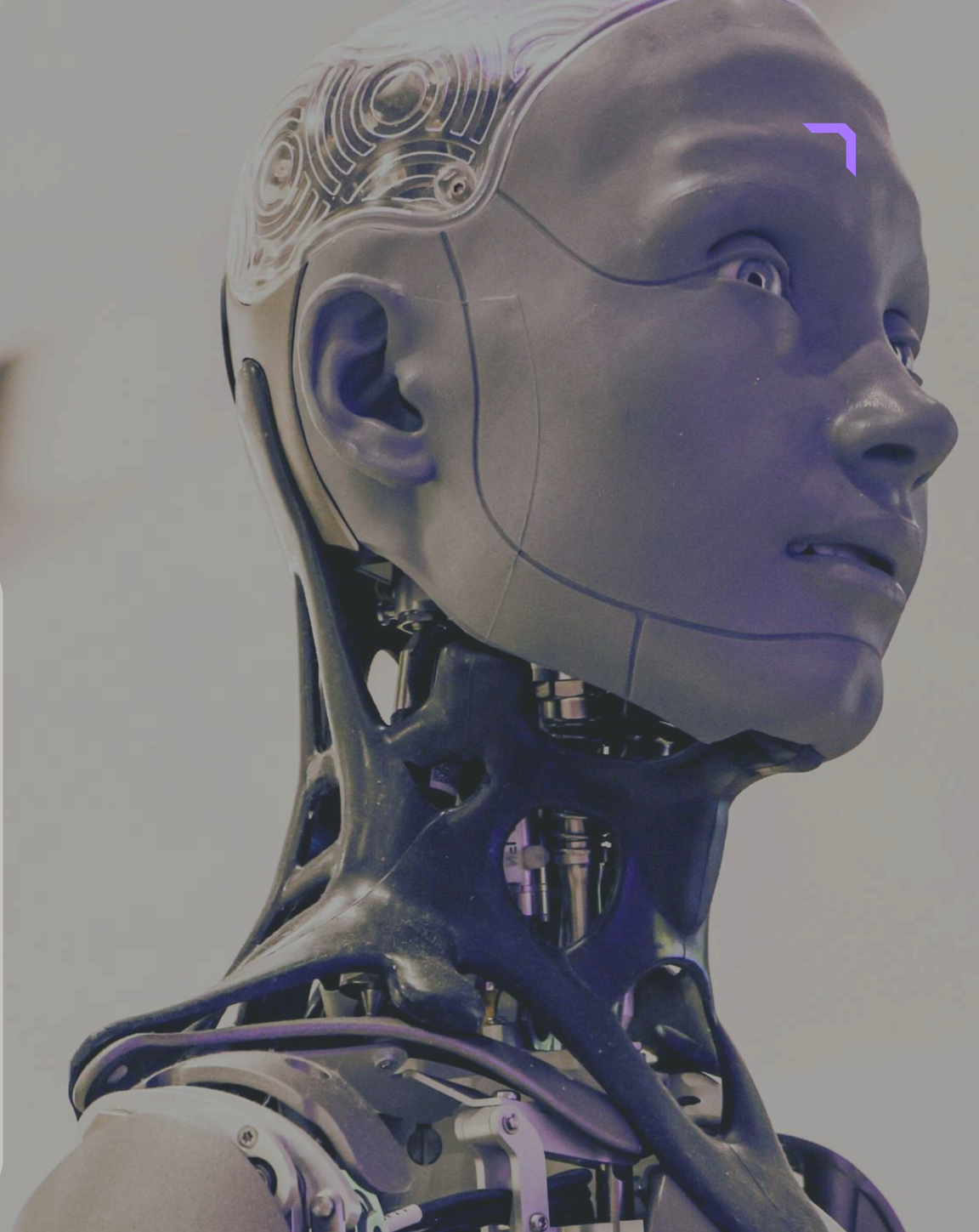
ReLU (Rectified Linear Unit)

- ReLU is one of the most widely used activation functions in deep learning.
- It converts negative values to 0.
- It keeps positive values unchanged.

Formula

$$ReLU$$

$$f(x) = max\ (0, x)$$



Activation Functions

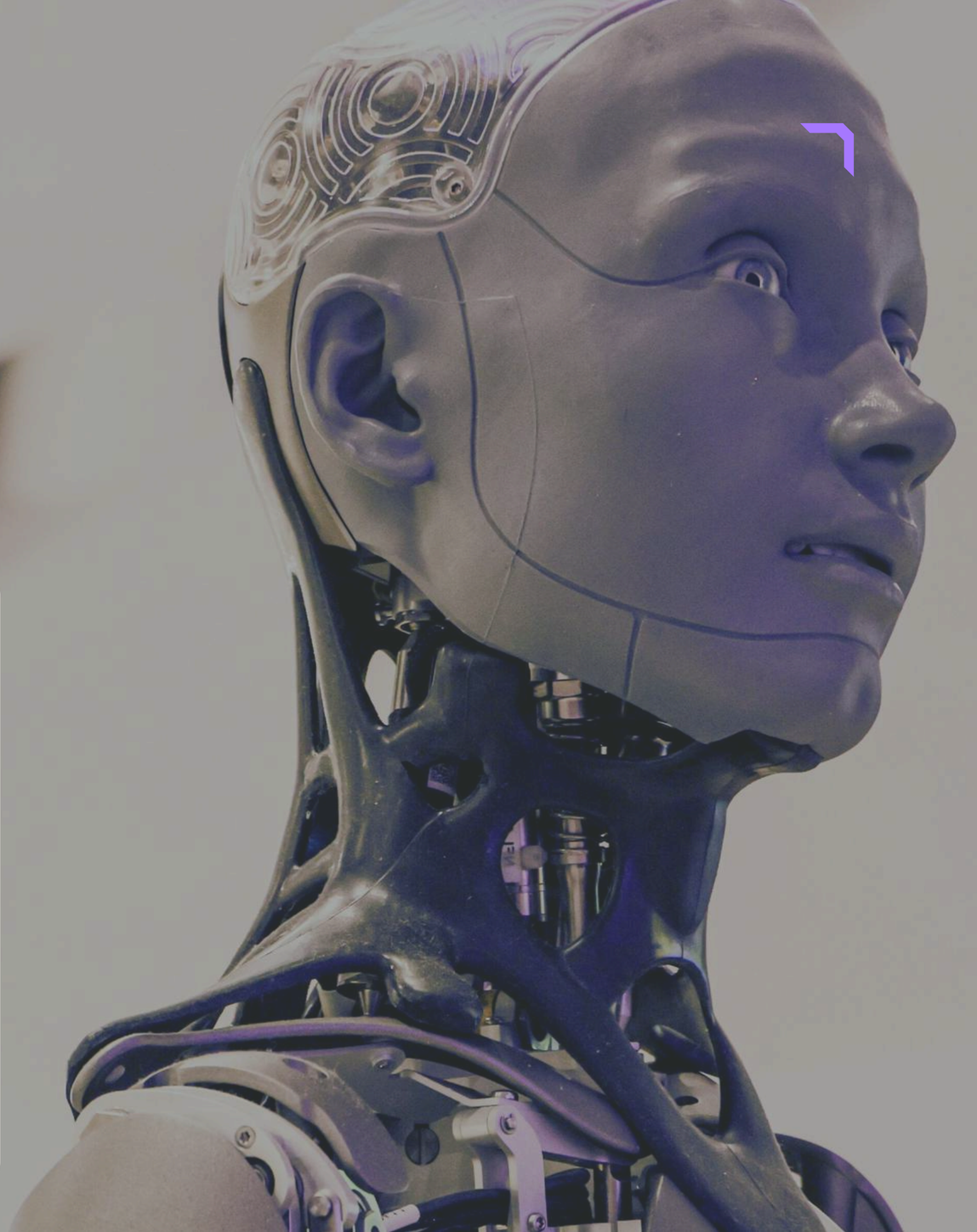
ReLU (Rectified Linear Unit)

Examples

- If $x=3 \rightarrow f(x)=3$
- If $x=-5 \rightarrow f(x)=0$

Advantages

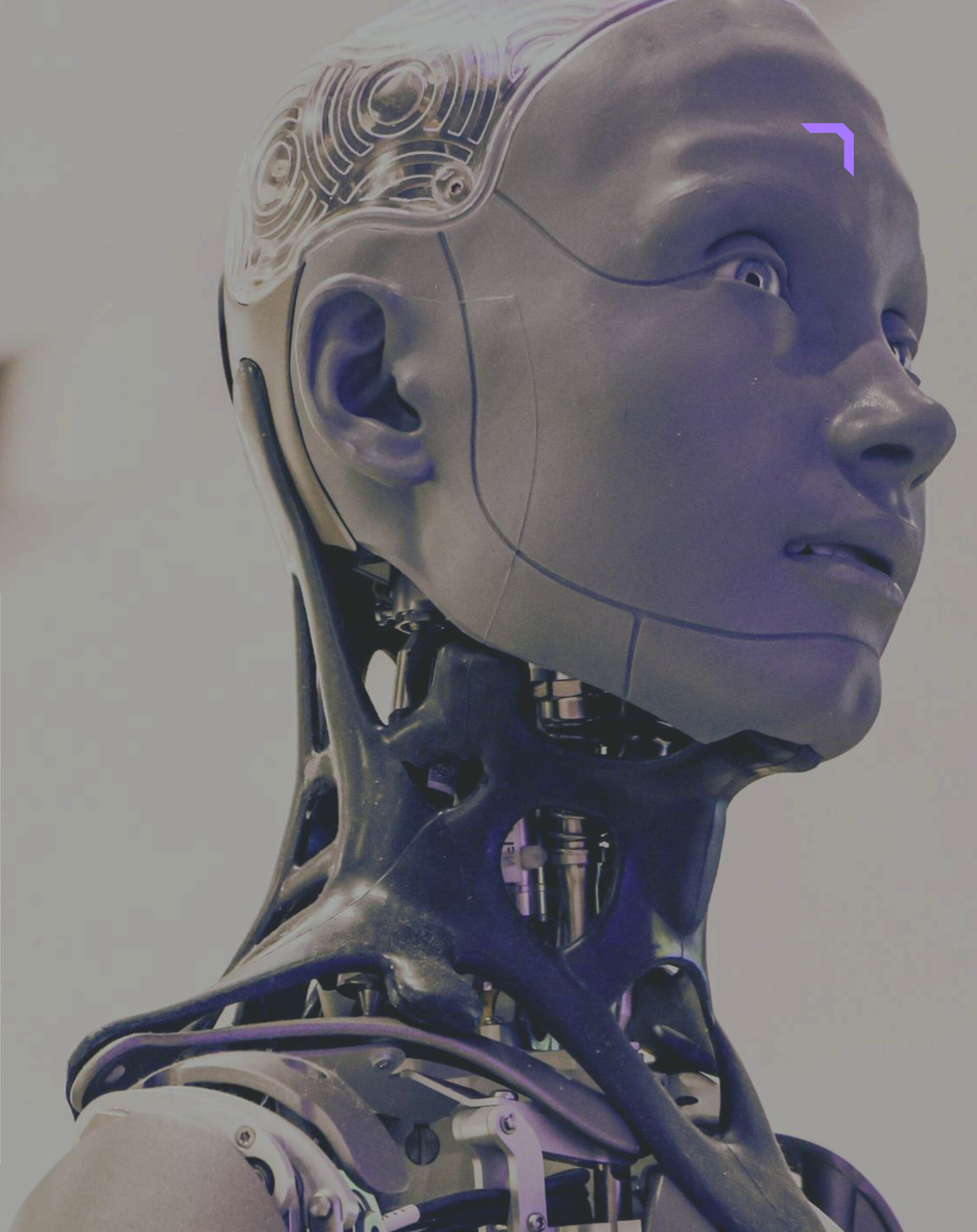
- Simple and fast to compute
- Helps in training deep neural networks efficiently



Loss Functions

Definition

- A Loss Function is used to measure how well a neural network's predictions match the actual values.
- It calculates the error between predicted output and actual output.
- During training, the loss function helps in adjusting the model's weights.
- The goal of training is to minimize the loss.

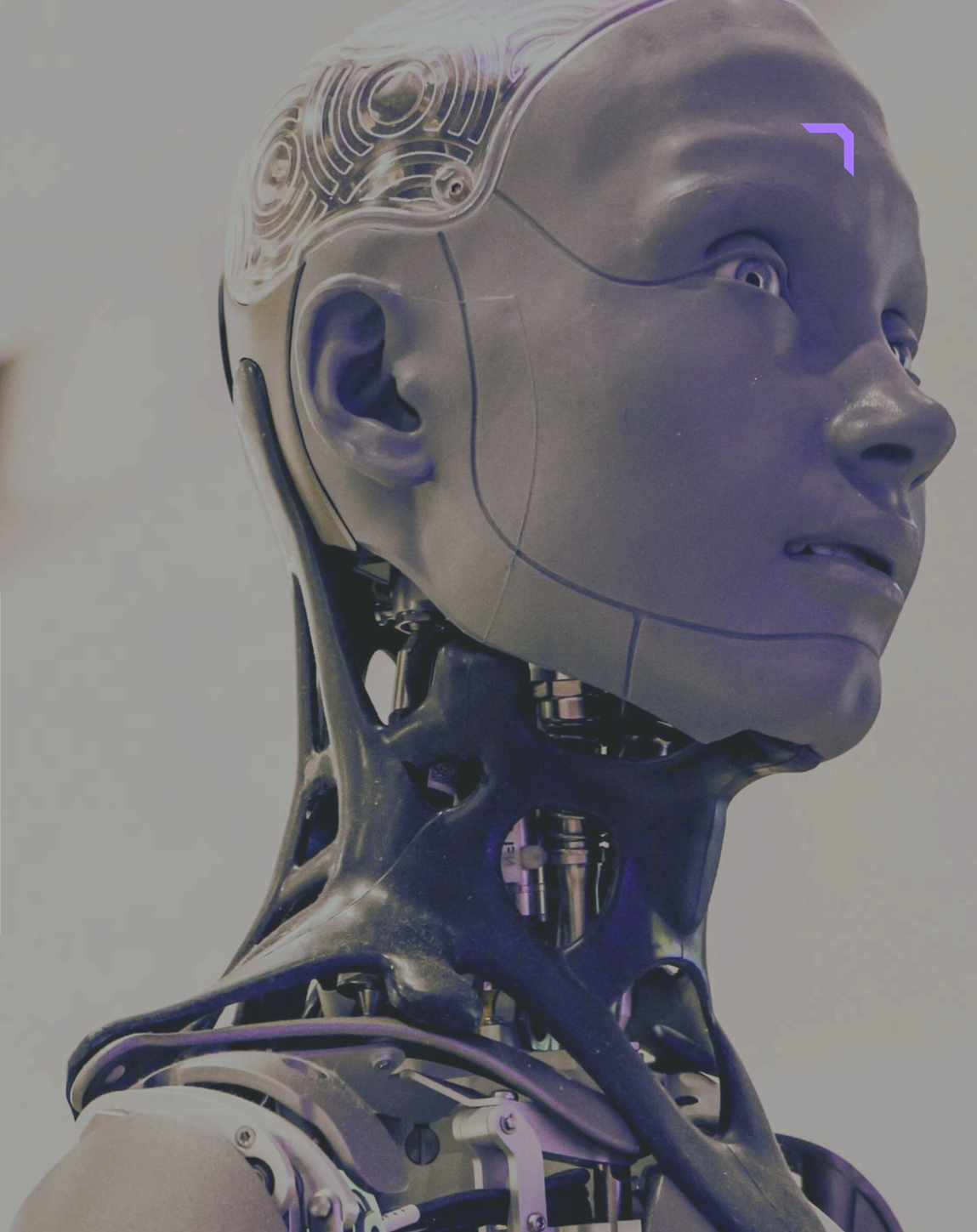


Loss Functions

Definition

Common Loss Functions

- Mean Squared Error (MSE)
- Mean Absolute Error (MAE)
- Binary Cross Entropy
- Huber Loss



Loss Functions

Mean Squared Error (MSE)

- Used mainly for regression problems.
- It calculates the average of squared differences between actual and predicted values.
- Squaring the error penalizes larger errors more strongly.
- Formula

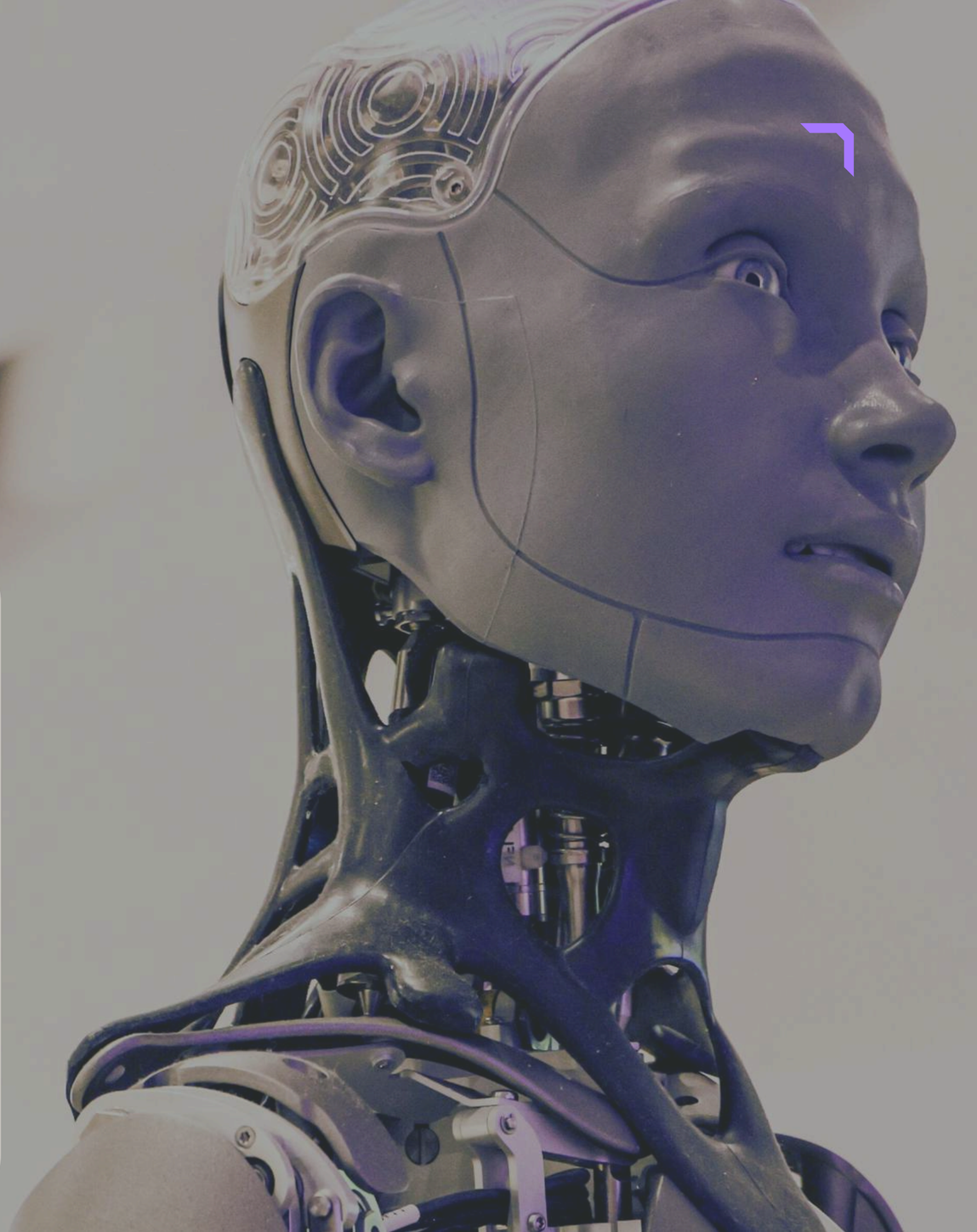
$$\mathrm{MSE} = \frac{1}{n}\sum(Y_i - \hat{Y}_i)^2$$

Loss Functions

Mean Squared Error (MSE)

Example

- Actual value = 5
- Predicted value = 4
- Error = (5−4)^2=1
- Larger errors get more penalty, helping the model reduce big mistakes.



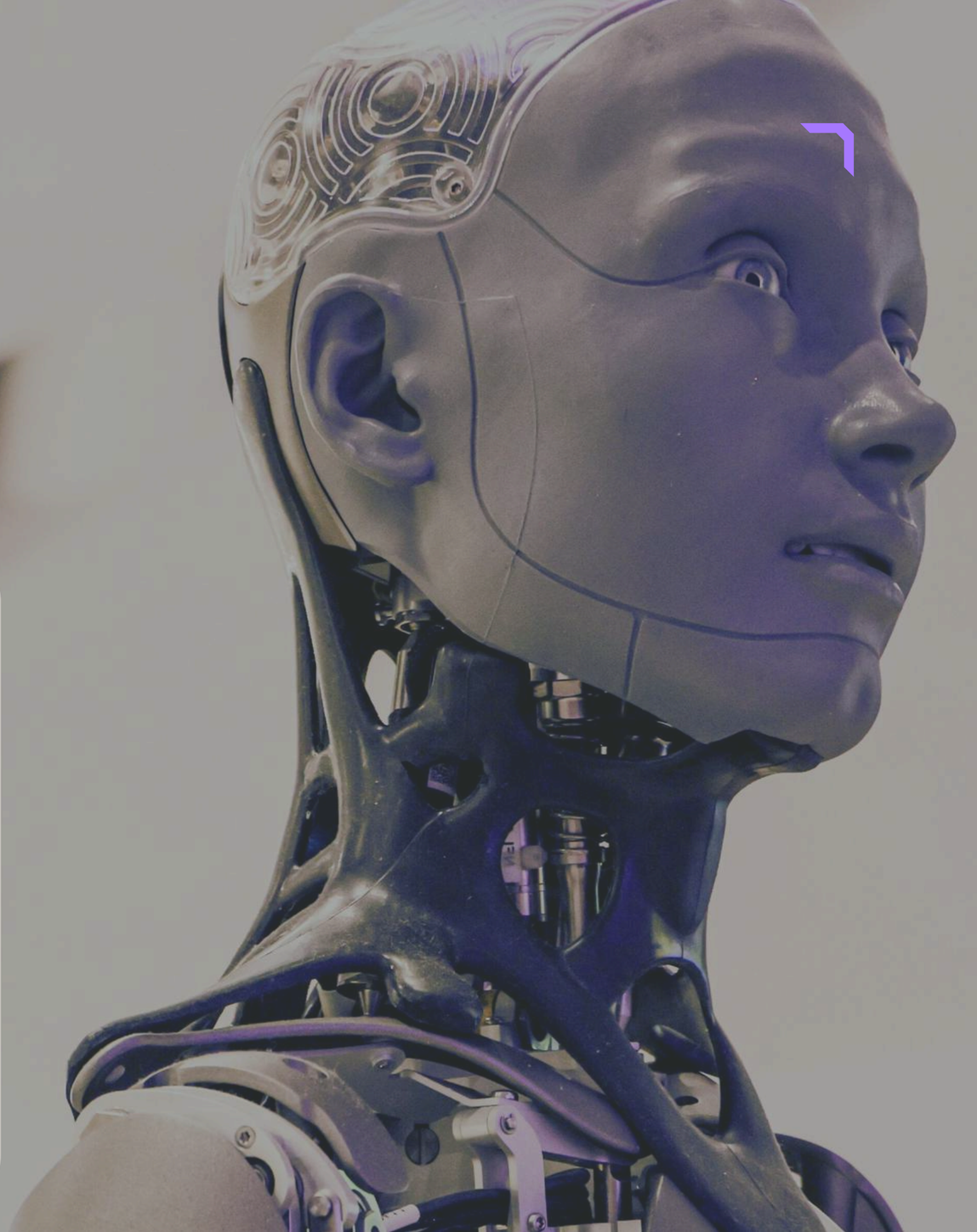
Loss Functions

Binary Cross Entropy

- Used for binary classification problems.
- Measures the difference between predicted probability and actual label (0 or 1).

Formula

$$Loss = -[y\log(p) + (1-y)\log(1-p)]$$



Loss Functions

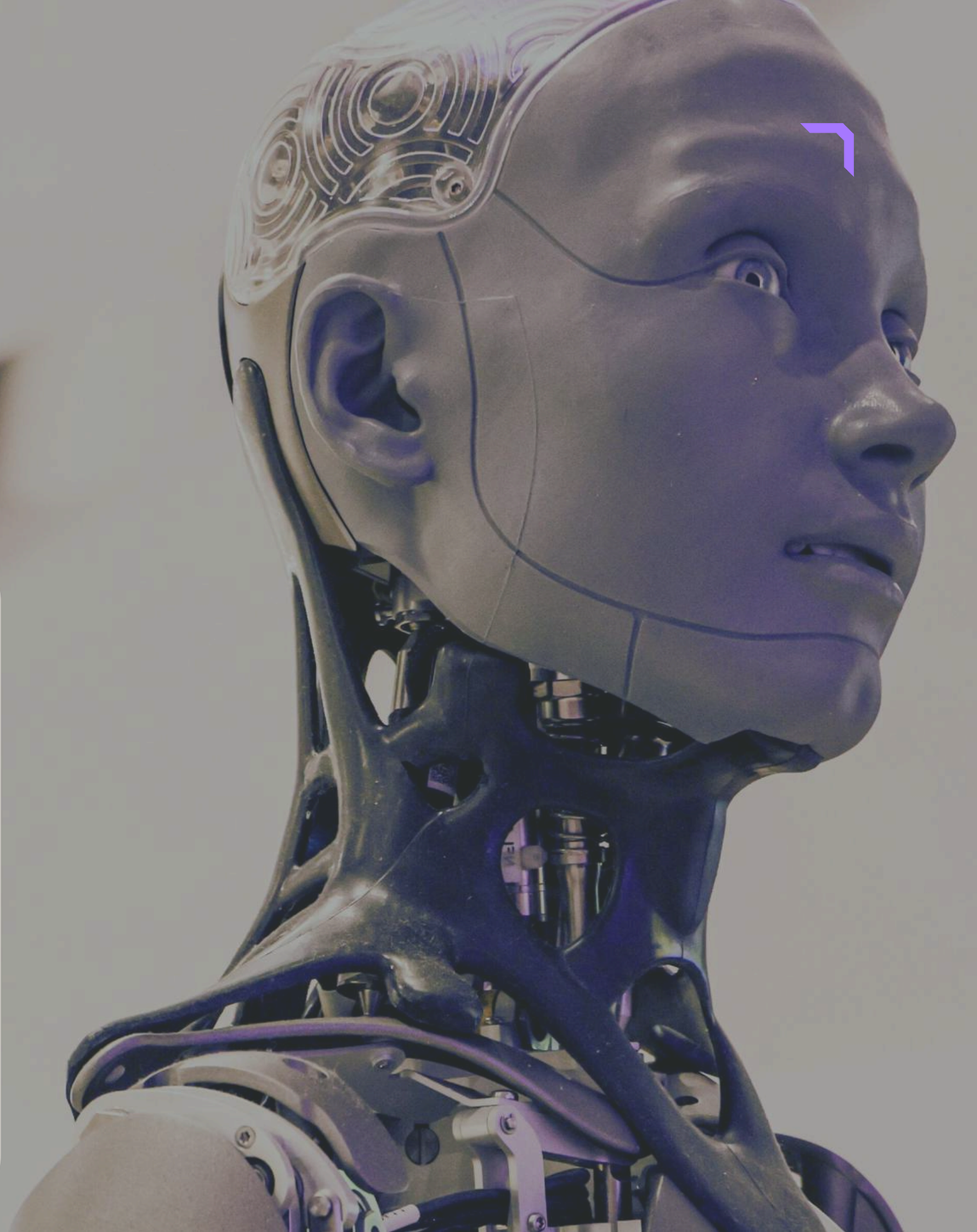
Binary Cross Entropy

Where:

- y = actual label (0 or 1)
- p = predicted probability

Example

- Actual label = 1
- Predicted probability = 0.8
- Loss will be small, meaning the prediction is good.



Hyperparameters

Definition

- Hyperparameters are settings that control how a neural network learns.
- They are not learned from data.
- They are set before the training process starts.
- Choosing the right hyperparameters helps improve model performance.

Hyperparameters

Important Hyperparameters

1. Learning Rate
   - Controls how much the weights are updated during training.
   - High learning rate → Faster learning but may overshoot the optimal solution.
   - Low learning rate → Slower but more stable learning.

Hyperparameters

Important Hyperparameters

2. Batch Size

- Defines the number of training samples processed at once.
- Small batch size → Faster updates but more noise.
- Large batch size → More stable training but requires more memory.

Hyperparameters

Important Hyperparameters

3. Epochs

- An epoch is one complete pass of the entire dataset through the neural network.
- More epochs allow the model to learn better.
- Too many epochs may cause overfitting.

Hyperparameters

Important Hyperparameters

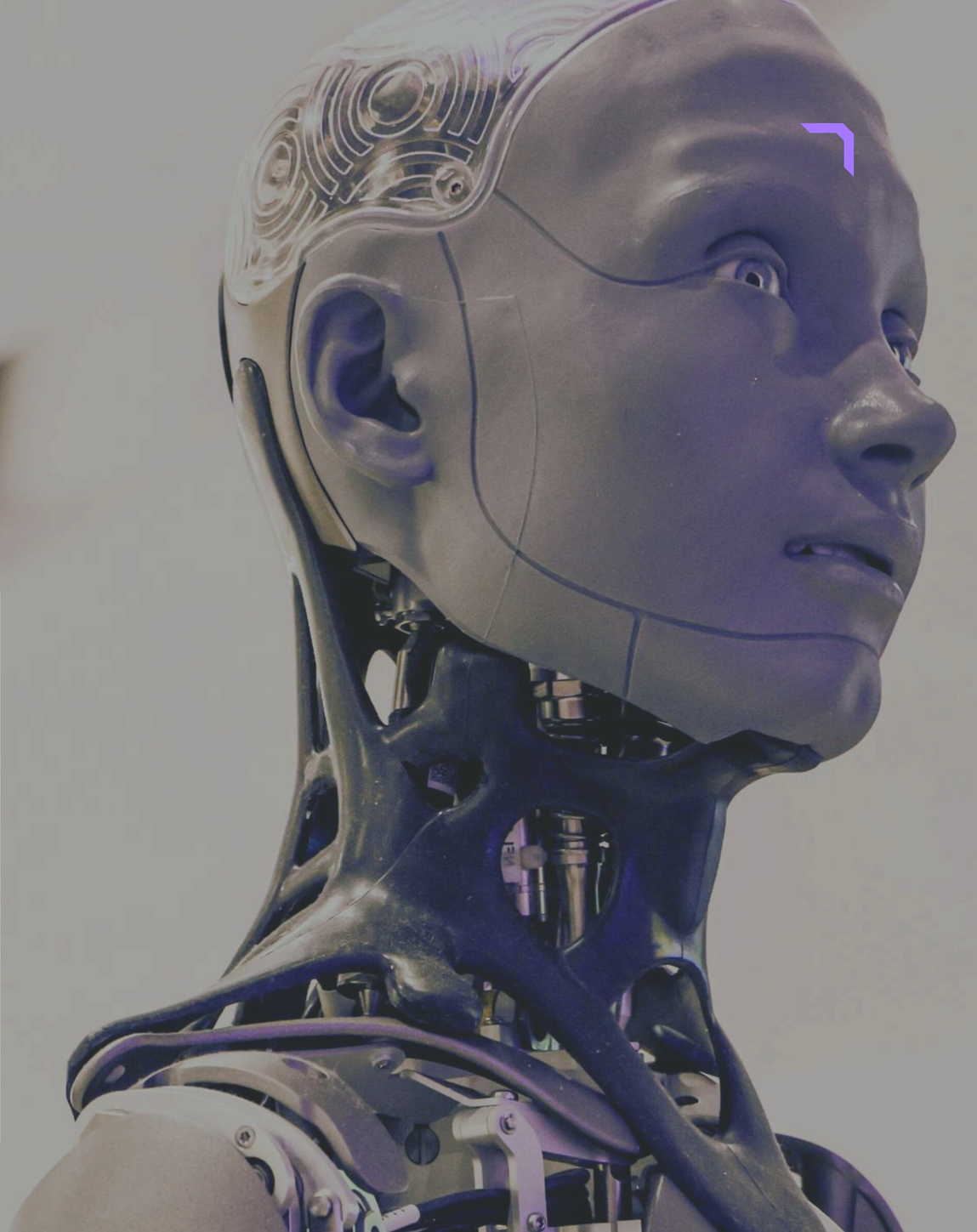
4. Dropout Rate

- A technique used to prevent overfitting.
- Randomly turns off some neurons during training.
- Helps the model generalize better.

Sentiment Analysis

Definition

- Sentiment Analysis is a Natural Language Processing (NLP) technique.
- It is used to determine the emotional tone behind a piece of text.
- It helps identify whether the text expresses positive, negative, or neutral sentiment.
- It is widely used in business, social media monitoring, and customer reviews.



Sentiment Analysis

Working of Sentiment Analysis

1. Data Collection
   - Text data is collected from sources such as:
   - Social media
   - Customer reviews
   - Surveys
   - Online feedback

2. Preprocessing
   - The text is cleaned by removing:

Sentiment Analysis

Working of Sentiment Analysis

- Stop words
- Special symbols
- Unnecessary characters

- This step prepares the data for analysis.

3. Feature Extraction

- Important words and phrases are identified.
- Common techniques used:
  - Bag of Words (BoW)
  - TF-IDF

Sentiment Analysis

Working of Sentiment Analysis

4. Classification

- A machine learning or deep learning model is used to classify sentiment.
- Examples of models:
  - Naive Bayes
  - Support Vector Machine (SVM)
  - LSTM
  - Transformers

Sentiment Analysis

Working of Sentiment Analysis

Examples

- "The food at this restaurant was delicious and the staff were very friendly."
- → Positive Sentiment
- "The delivery was very late and the product quality was poor."
- → Negative Sentiment
- "The laptop works fine, but it is nothing extraordinary."

Sentiment Analysis

Working of Sentiment Analysis

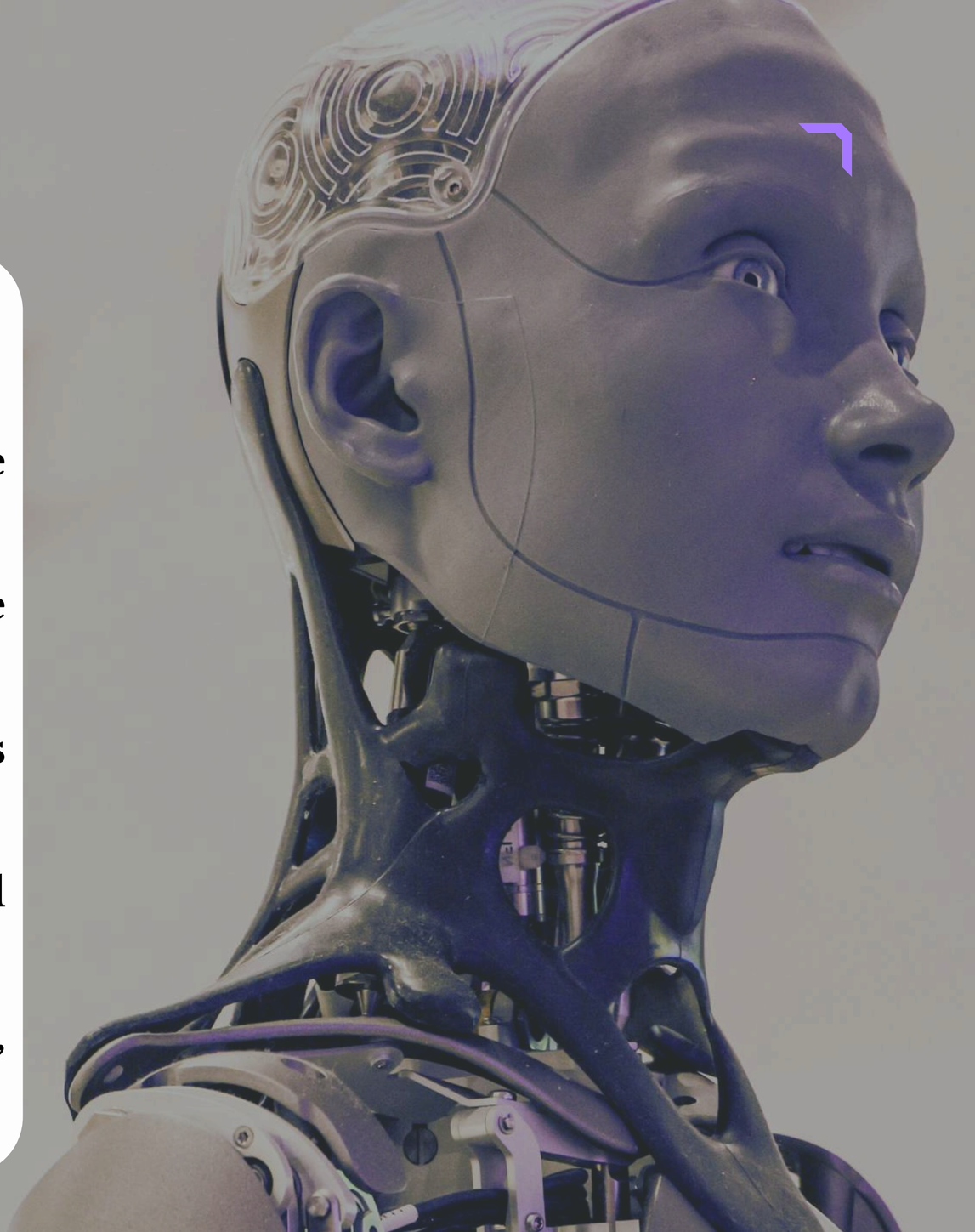
- → Neutral Sentiment

Applications of Sentiment Analysis

1. Social Media Monitoring
   - Companies analyze opinions on platforms like Twitter, Facebook, and Instagram.
2. Customer Feedback Analysis
   - Businesses understand customer satisfaction through product reviews.

PyTorch

- PyTorch is an open-source deep learning framework.
- It was developed by Meta (Facebook).
- It is widely used for building and training machine learning and deep learning models.
- PyTorch is popular because of its flexibility and ease of use.
- It uses a dynamic computation graph, which allows developers to modify models during runtime.
- It supports GPU acceleration, which makes model training faster.
- PyTorch is mainly used in research, computer vision, and Natural Language Processing (NLP) applications.

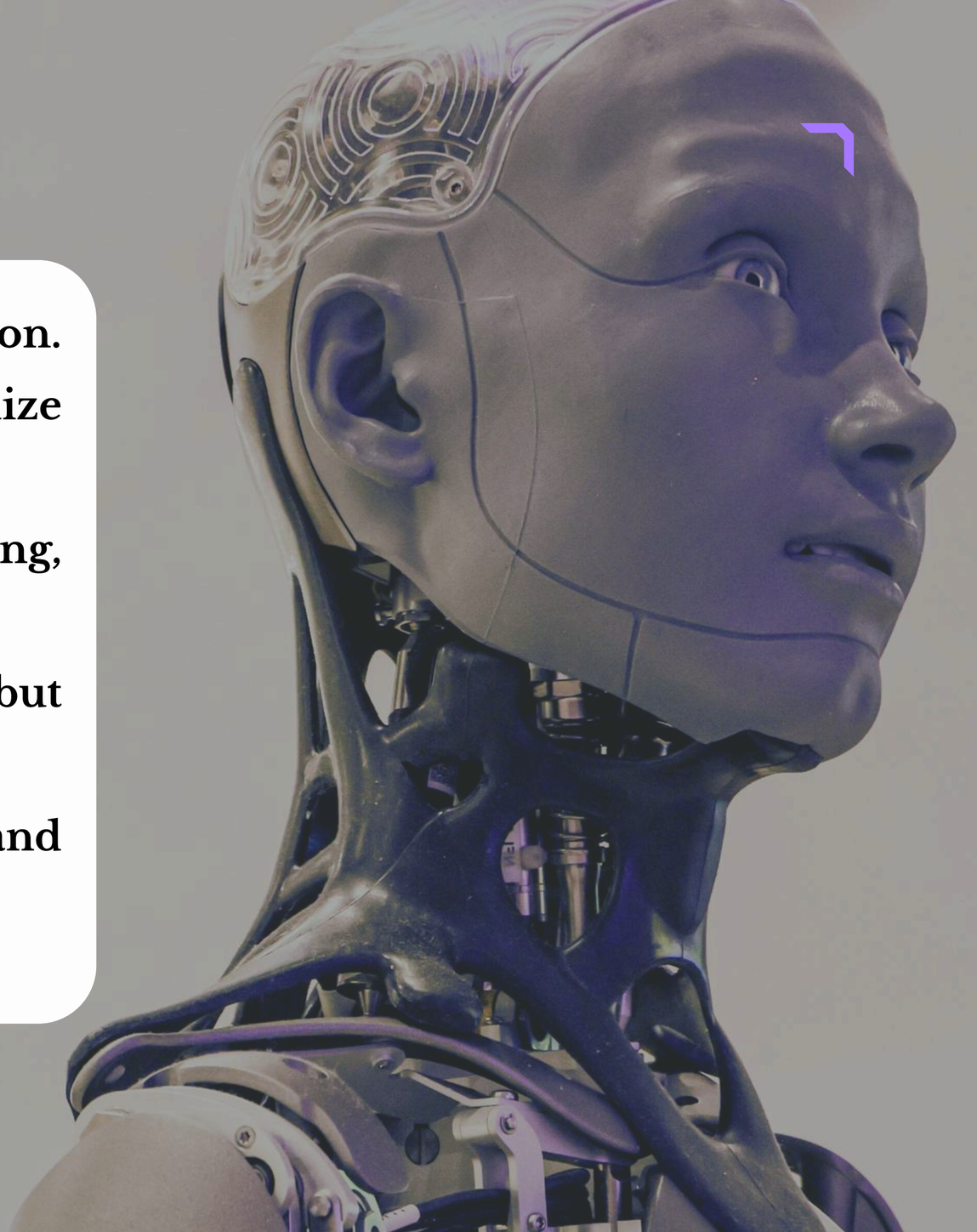


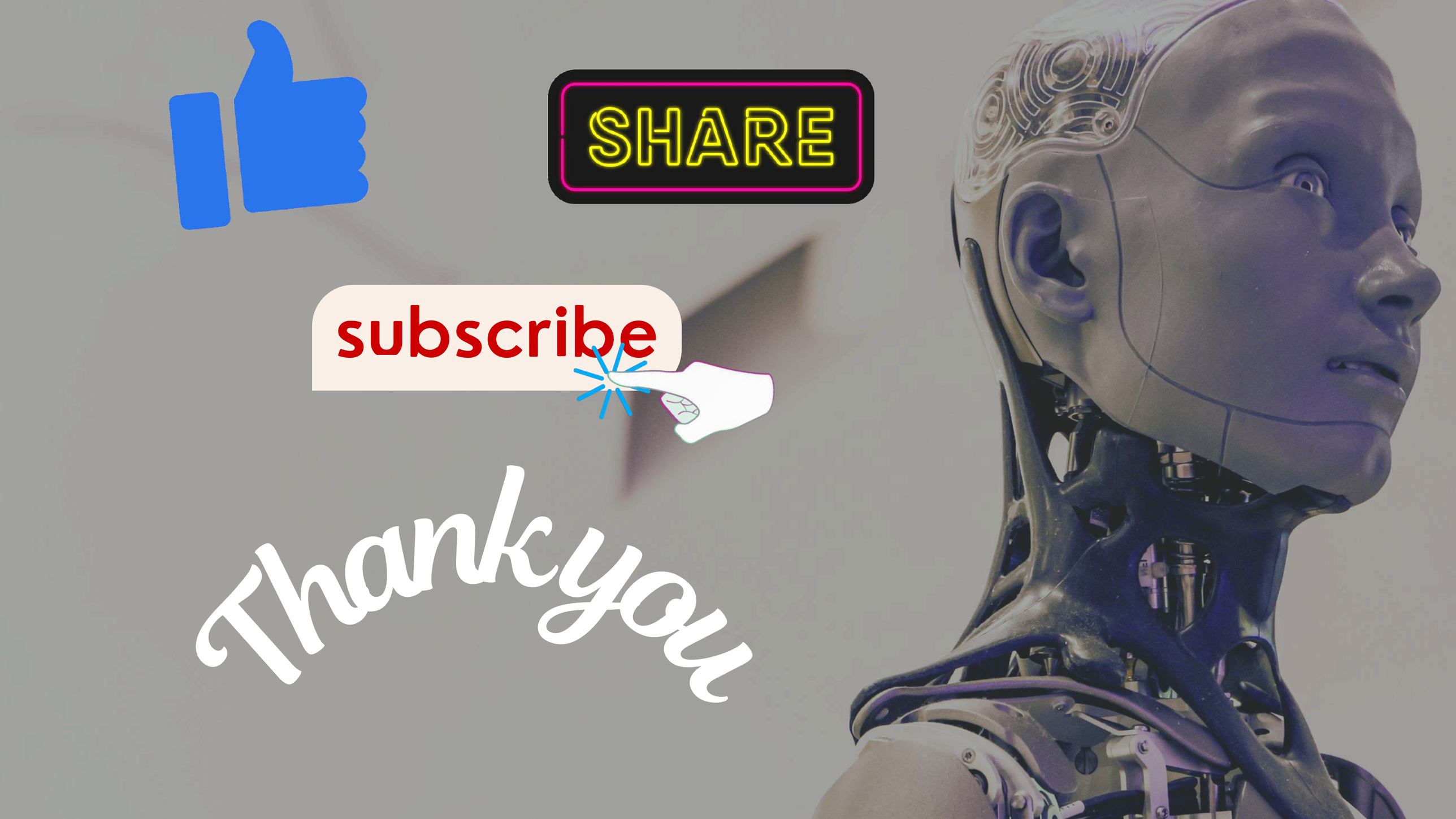
Google Colab

- Google Colab (Colaboratory) is a cloud-based platform.
- It allows users to write and run Python code in a web browser without installing any software.
- It provides free access to GPUs and TPUs, which helps in training deep learning models faster.
- Colab is based on Jupyter Notebook.
- It supports popular libraries such as PyTorch, TensorFlow, NumPy, and Pandas.
- It is widely used by students, researchers, and developers for experiments and quick prototyping.

Jupyter Notebook

- Jupyter Notebook is an open-source web application.
- It allows users to write and execute code, visualize data, and add explanations in a single document.
- It is widely used in data science, machine learning, and research.
- Jupyter supports many programming languages, but it is mainly used with Python.
- It allows combining code, text, equations, and visualizations in one place.




SHARE
subscribe
Thankyou